Taming Multi-Dimensional Skew in Sparse Matrix Multiplication with Contention-Aware Scheduling [Technical Report]

Wentao Huang
National University of Singapore
huangwentao@u.nus.edu

Qiming Huang
National University of Singapore
huang_qiming@u.nus.edu

Yunhong Ji
Renmin University of China
jiyunhong@ruc.edu.cn

Mian Lu
4Paradigm Inc.
lumian@4paradigm.com

Yuqiang Chen
4Paradigm Inc.
chenyuqiang@4paradigm.com

Bingsheng He
National University of Singapore
dcsheb@nus.edu.sg

Kian-Lee Tan
National University of Singapore
dcstankl@nus.edu.sg

ABSTRACT

Sparse general matrix-matrix multiplication (SpGEMM) is a key kernel in graph analytics, relational queries, and data mining. Yet, its performance remains a significant challenge, due to the inherent sparsity and, more critically, the prevalent skew patterns often found in real-world workloads. This paper investigates this problem and demonstrates that an unbalanced workload distribution is the main culprit, harming performance through two co-existing root causes: first, *multi-dimensional skew*, where imbalance across computation cost, output density, and data locality defies any single balancing metric; and second, *cache contention*, where dense substructures induce severe on-chip interference.

To that end, we propose adaptive morsel scheduling (AMS), a dynamic scheduling framework that decouples skew dimensions via fixed-result-size morsels and integrates contention-aware load balancing (CALB) and data-conscious simultaneous multithreading activator (SMTact) to alleviate cache-level contention, alongside a hybrid accumulation mechanism (HYB) that adaptively selects the best accumulator per morsel. Evaluations across matrix squaring, relational queries, graph analytics, and other sparse matrix kernels show that AMS significantly outperforms state-of-the-art methods on skewed workloads while maintaining comparable performance on non-skewed ones. These gains, along with improved bandwidth utilization, remain valid across a wide range of workloads and different hardware systems.

PVLDB Reference Format:
Wentao Huang, Qiming Huang, Yunhong Ji, Mian Lu, Yuqiang Chen, Bingsheng He, and Kian-Lee Tan. Taming Multi-Dimensional Skew in Sparse Matrix Multiplication with Contention-Aware Scheduling [Technical Report]. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/fukien/casmm.

Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

1 INTRODUCTION

Matrix multiplication is a fundamental computational kernel, and its sparse variant, sparse general matrix-matrix multiplication (SpGEMM), is especially important in practice. The reason is that sparse matrices are a natural model for real-world graph data, which makes SpGEMM a core building block for applications such as graph analytics, entity resolution, and machine learning [38, 57, 69, 72, 78, 93, 114]. Recently, SpGEMM has emerged as a transformative method for processing relational join-aggregate and project queries [32, 49–51, 54, 90–92] [1] . By treating relational tables as algebraic sparse matrices, entire query pipelines can be executed as unified algebraic operations. This enables aggregation or projection to be performed *on-the-fly* alongside the join, effectively bypassing the prohibitive costs of materializing excessive intermediate results (see Figure 1 for a comparison) . In all such cases, the performance of the application is dictated by the efficiency of the underlying parallel SpGEMM execution. Reflecting its critical role, numerous studies have investigated SpGEMM optimization, reaching a broad consensus that it is a fundamentally bandwidth-intensive kernel where throughput is ultimately governed by available memory bandwidth [45, 76].

Though it is bandwidth-bound, running SpGEMM on a high-bandwidth platform does not necessarily translate to higher throughput [2] . This is primarily attributed to the *load imbalance* caused by the widely observed skew patterns in real-world workloads [15, 28, 60, 65, 85, 101, 113]. For instance, in a typical OLAP ClickHouse benchmark [87], the top 5% of users generate 24.4% (47.8 million) of all clicks, while the median number of clicks per user is only 2. Similarly, in a common PageRank workload [51, 107], the top 1.8% of pages receive more incoming links than the remaining 98.2% combined. From a matrix perspective, such skewed key frequencies and power-law graph degrees translate into highly irregular nonzero

[1] The paper focuses on join-aggregate queries, as they produce large intermediate cardinalities that stress the SpGEMM kernel; the same framework applies to project queries, which share the same algebraic formulation.
[2] While SpGEMM above a certain density threshold can be compute-limited, the growing compute-bandwidth gap [41, 67, 82, 98] makes compute progressively less likely than bandwidth to be the primary bottleneck.

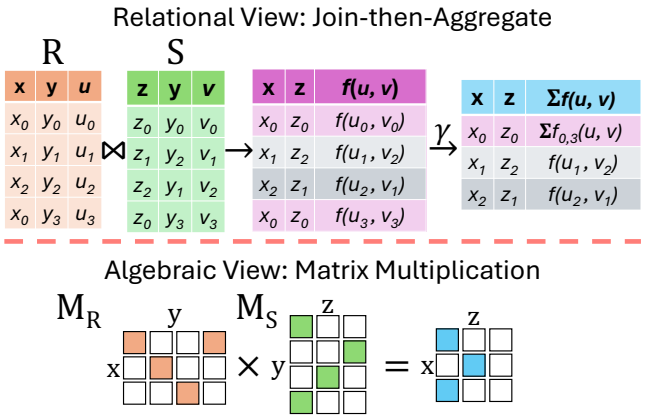


Figure 1: Comparing two approaches for evaluating a join-aggregate query, " $SELECT\ x, z, SUM(f(u,v))\ FROM\ R, S\ WHERE\ R.y = S.y\ GROUP\ BY\ x, z$ ", where f denotes a mathematical function. The "relational view" (top) joins tables $R(x, y, u)$ and $S(z, y, v)$ on attribute y followed by an aggregation, a process that produces excessive intermediate results, such as $(x_0, z_0, f(u_0, v_0))$ and $(x_0, z_0, f(u_3, v_3))$, which are ultimately aggregated into one final group-by result (shaded in light pink). The "algebraic view" (bottom) models the operation as a matrix product $M_R \times M_S$ where y serves as the shared dimension to compute the aggregate result directly, bypassing the generation of costly intermediate results.

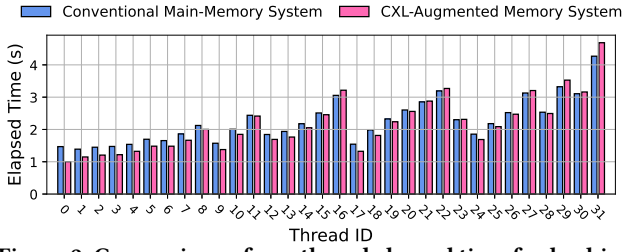


Figure 2: Comparison of per-thread elapsed time for hashing-based SpGEMM [76] on a G500 skewed matrix (scale=17, *edgefactor*=16) [23] between a conventional main-memory system (28.50 GB/s bandwidth) and a CXL-augmented system (46.50 GB/s bandwidth) (see § 4.1 for experimental setups).

structures, resulting in significant load imbalance. When running SpGEMM on these workloads, a small subset of cores becomes overloaded while others remain underutilized [38, 45, 76]. This not only limits compute efficiency but also prevents full utilization of the available memory bandwidth, leaving system resources idle, and therefore, rendering high-bandwidth platforms less effective.

Moreover, the challenge of addressing load imbalance in skewed workloads is particularly critical and is further exacerbated by the "memory scaling wall", where memory technology advancements lag behind those of processing units [82, 98]. This trend results in decreasing memory bandwidth and capacity per processing core [53, 105], which further compounds the load imbalance problem for two reasons:

First, decreasing bandwidth-per-core has an acutely negative impact on SpGEMM workloads with load imbalance, where a few straggling threads drag down performance. Since these threads largely rely on core-level bandwidth, their overall performance is inevitably reduced due to the insufficient bandwidth-per-core caused by the "memory scaling wall."

Second, the prevalent approach to compensating for decreasing bandwidth-per-core, adopting heterogeneous memory expansion technologies like CXL [52, 64, 105, 106], can inadvertently amplify this problem. Since CXL-attached memory typically exhibits higher access latency than local DRAM [3] , straggling threads that run disproportionately long are more exposed to CXL's latency penalty, further widening the gap between fast and slow threads. Figure 2 illustrates this using a hashing-based SpGEMM algorithm [76] on a skewed workload, where per-thread runtimes vary due to load imbalance. [4] Because total runtime is governed by the slowest thread, the disproportionate latency on stragglers negates the aggregate bandwidth gain, yielding worse overall performance than pure DRAM. This counterintuitive result is also reported in existing CXL studies [70, 71, 94, 97]. As a result, addressing load imbalance is becoming more imperative than ever.

However, addressing load imbalance in SpGEMM is not trivial. While numerous strategies have been proposed [38, 45, 79], we maintain that existing approaches fall short due to two co-existing root causes: *multi-dimensional skew* and *cache contention*, both of which not only hinder load balancing but also degrade overall performance.

First, existing methods fail to accommodate the *multi-dimensional skew* inherent in sparse matrices. Current methods rely on static schemes that balance along a single metric (e.g., flop count) to minimize scheduling overhead. However, skew manifests across multiple dimensions, including computation cost, materialization overhead, and data locality, making it impossible for any single static metric to achieve balanced execution across all threads.

Second, in skewed workloads, existing approaches suffer from significant cache contention, induced by dense substructures that are co-scheduled together. In SpGEMM, irregular sparsity forces practitioners to cache row-wise intermediates to reduce random access costs. However, dense substructures within skewed matrices incur excessive irregular random entry access, which creates heavy cache pressure and even induces significant cache contention, which has long been overlooked as a key performance factor. This cache contention is particularly pronounced under simultaneous multithreading (SMT), which aims to increase computation parallelism but worsens the interference.

In this paper, we address these challenges through a novel approach, *adaptive morsel scheduling (AMS)*. In contrast to de facto methods, AMS revisits *dynamic scheduling*, a strategy conducive to load balancing that is often dismissed due to concerns regarding

[3]Since DRAM is the primary technology used in memory manufacturing, we use the terms "DRAM" and "memory" interchangeably throughout this paper.

[4]We use the following notation for R-MAT graphs [23]: **ER**, **SSCA**, and **G500** denote non-skewed Erdős–Rényi, skewed SSCA, and highly skewed Graph500 graphs, respectively; *scale* sets the number of rows to 2^{scale}, and *edgefactor* sets the average nonzeros per row.

scheduling overhead and poor performance [45, 76]. Through analysis (§ 2.2), we find that task granularity is the primary performance factor: a granularity that is too fine fails to preserve spatial locality and therefore suffers from excessive cache thrashing. Hence, we employ a morsel-wise dynamic scheduling strategy, where a "morsel"[5] is defined as a basic scheduling unit of several consecutive matrix rows. AMS ensures a morsel is large enough to maintain spatial locality, thereby greatly mitigating cache thrashing, and more importantly achieves load balance on skewed workloads through dynamic scheduling, effectively preventing computational idling and improving memory bandwidth utilization.

To alleviate cache contention, AMS introduces two lightweight and generalizable techniques: *contention-aware load balancing (CALB)*, which mitigates cache interference by adaptively scheduling dense morsels, and *data-conscious simultaneous multithreading activator (SMTact)*, which selectively enables SMT based on workload contention. These optimizations are simple in design, yet effective in practice, and easy to generalize to other sparse matrix kernels (e.g., SpMM and SpMV). Importantly, their performance benefits remain effective across heterogeneous bandwidth-boosted systems (e.g., CXL systems), ensuring that latency heterogeneity does not cause performance degradation.

Furthermore, the morsel-wise scheduling strategy enables a *hybrid accumulation mechanism (HYB)* for SpGEMM, where different accumulation algorithms can be selectively applied to morsels based on their intrinsic characteristics, thereby achieving more efficient execution and enhanced overall SpGEMM performance.

The contributions of this paper are summarized as follows:

(1) We examine and characterize two co-existing root causes of performance degradation in skewed SpGEMM workloads: *multi-dimensional skew*, where no single static metric can balance all threads, and *cache contention*, where dense substructures induce severe on-chip interference that existing methods overlook.
(2) We develop a novel morsel-wise dynamic scheduling framework, adaptive morsel scheduling (AMS), that systematically addresses both *multi-dimensional skew* and *cache contention* in SpGEMM. AMS decouples skew dimensions through fixed-result-size morsels, integrates two key techniques, contention-aware load balancing (CALB) and data-conscious simultaneous multithreading activator (SMTact), to alleviate on-chip cache interference, and enables hybrid accumulation mechanism (HYB) for morsel-adaptive accumulation. Our approach is straightforward in design, yet novel and generalizable, and effective in practice.
(3) We evaluate AMS across a wide range of workloads, including matrix squaring, relational join-aggregate and join-project queries, graph analytics, and other sparse kernels, on both a main-memory and a CXL-augmented system, and show that it performs on par with state-of-the-art methods on non-skewed workloads while significantly outperforming them on skewed ones, addressing both computational idling and bandwidth underuse.

The rest of the paper is organized as follows. We first present the background and motivation in § 2, followed by a description of AMS in § 3. We then show the experimental results in § 4 and discuss related work in § 5. Finally, we conclude the paper in § 6.

[5]We adopt the term from Leis et al. [66], where it denotes a fixed-size fragment of input data.

Algorithm 1 Parallel Gustavson's Algorithm for SpGEMM

Input: CSR matrices $A \in \mathbb{R}^{m \times k}$ and $B \in \mathbb{R}^{k \times n}$
Output: CSR matrix $C \in \mathbb{R}^{m \times n}$
for parallel for $i \leftarrow 0$ to $m-1$ **do** ▷ Parallel over rows of A
    Initialize empty `accumulator` for row i
    for all $(k, A_{ik}) \in$ row i of A **do**
        for all $(j, B_{kj}) \in$ row k of B **do**
            if $j \in$ `accumulator` **then**
                `accumulator[`j`]` += $A_{ik} \cdot B_{kj}$
            else
                `accumulator[`j`]` $\leftarrow A_{ik} \cdot B_{kj}$
            end if
        end for
    end for
    Store `accumulator` as row i of C
end for

2 BACKGROUND AND MOTIVATION

We first review state-of-the-art SpGEMM algorithms, then analyze how multi-dimensional skew and cache contention degrade performance under skewed workloads, and finally revisit dynamic scheduling as a promising solution, motivating our AMS approach.

2.1 The SpGEMM

SpGEMM computes the product $C = AB$, where A, B, and C are all sparse matrices [6] . This form has proven effective across diverse large-scale workloads, including scientific computing, graph analytics, and more recently relational query processing [49, 75, 76, 91]. To reduce memory footprint at scale, sparse matrix formats are widely used, with compressed sparse row (CSR) being the most common representation in existing works [13, 21, 43, 56, 84]. However, the irregular and input-dependent sparsity patterns of sparse matrices, combined with irregular memory access patterns, make SpGEMM a fundamentally bandwidth-intensive workload, where performance depends critically on how effectively memory bandwidth is sustained across cores. Moreover, predicting the number and locations of nonzeros in the output matrix is often intricate, making memory pre-allocation challenging.

To address this pre-allocation challenge, SpGEMM is typically divided into two phases: a symbolic phase, which determines the nonzero structure of the output, and a numeric phase, which performs the actual computation based on that structure. In addition, the irregular nature of sparsity makes it difficult to efficiently gather and accumulate scattered contributions to each output entry. To tackle this, prior work has introduced a variety of accumulator designs to aggregate partial results during computation, including sorted variants such as heaps [75, 76], SPA [42, 47], and expand-sort-compress (ESC) [14, 30], as well as unsorted variants based on hash tables [48, 75, 76].

[6]Throughout this paper, we refer to A and B in $C = AB$ as the left-hand side and right-hand side matrices, respectively.

Building on these data representations, SpGEMM algorithms are commonly implemented using one of three formulations: inner-product [81], outer-product [3, 45], or row-row paradigm (Gustavson's algorithm) [47]. Among these, Gustavson's algorithm operates in a row-wise manner, allowing each row of the left-hand side matrix (A) to be computed independently, enabling straightforward thread-level parallelization without the need for inter-thread synchronization (see Algorithm 1). It is this high degree of parallelism, combined with its efficient memory access patterns, that has made Gustavson's algorithm a dominant formulation for SpGEMM across a wide range of computing platforms.

Following Gustavson's algorithm, several load-balancing techniques have been proposed [3, 5, 11, 19, 25, 100], with static scheduling based on a flop-count scheme emerging as the dominant strategy [75, 76]. This method partitions the rows of matrix A into contiguous regions, each assigned approximately the same number of floating-point operations (i.e., additions plus multiplications). To achieve this, a lightweight preprocessing pass scans matrices A and B to estimate the flop count for each output row of C without performing actual multiplications. Based on these estimates, rows are grouped into partitions such that each thread receives a workload with a nearly equal number of floating-point operations. This approach enables static, flop-count-balanced scheduling across threads while avoiding runtime synchronization.

2.2 The Flop-Count-Based Scheme Under Skew

Despite its simplicity and widespread adoption [21, 76], the flop-count-based scheme is fundamentally limited and fails to achieve effective workload balance under skewed patterns. We identify two root causes, an inherent ignoring of multi-dimensional skew and the performance penalty of overlooking cache contention, both of which hurt load balancing and degrade bandwidth utilization.

(1) Ignoring multi-dimensional skew. Skewed SpGEMM workloads can exhibit imbalance across multiple dimensions, such as submatrix density and row-wise nonzero (*nnz*) distribution. However, the flop-count-based scheme equalizes only the total number of floating-point operations, ignoring other forms of skew. As a result, one thread may receive a few dense rows (with high *nnz*), while another is assigned many sparse rows. Although these assignments may have similar total flop counts, their structural differences lead to divergent runtime behaviors. Dense rows, in particular, induce "clustering" [7] effects during processing, which manifest as temporal bursts of memory and cache pressure. These effects significantly degrade performance, and the extent of impact varies by accumulation strategy [30, 42, 75]. Therefore, the scheme is inherently unable to preserve these skew dimensions.

Additionally, thread-level partitions often exhibit different compression factors (cf) [8] , which leads to divergent trends in computation and materialization times, making it challenging to effectively balance these two workload components. Consider Figure 3: using Gustavson's algorithm, partition-0 (the upper two rows of A) requires 12 flop counts [9] , producing 6 nonzeros; partition-1 (the bottom three rows) requires 11 flop counts but produces 9 nonzeros. Despite both involving comparable amounts of computation, the second has larger output size, inducing more materialization cost.

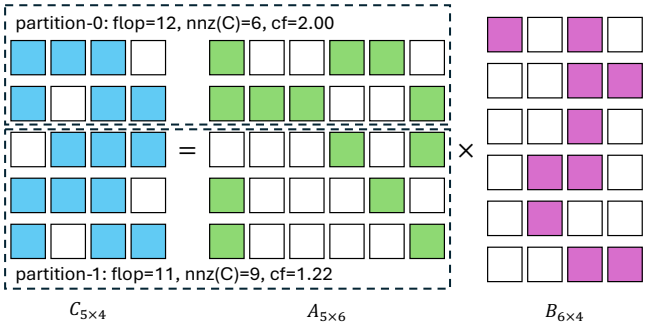


Figure 3: An illustration of $C = AB$ in SpGEMM (colored cells denote nonzero elements), where the two dashed boxes represent partitions with comparable flop counts but differing numbers of output nonzeros and compression factors.

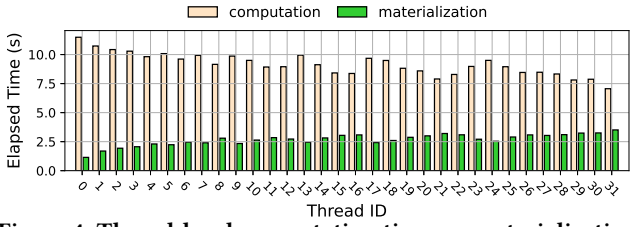


Figure 4: Thread-level computation time vs. materialization time for hashing-based SpGEMM on a G500 skewed matrix (scale=19, *edgefactor*=8).

Empirical data on a skewed workload confirms this disparity, as Figure 4 shows: materialization time varies across threads, with computation time also exhibiting thread-level variation, but in the opposite trend. Moreover, these complexities are further compounded by hardware variability: performance sensitivity to the relative costs of computation versus memory access differs considerably across hardware platforms. This architectural variance makes it exceedingly difficult for purely flop-count-based schemes to establish a load balance that is both universally effective, and readily generalizable across diverse machines. Thus, such schemes are inherently limited in delivering robust load balancing.

(2) Overlooking cache contention. Existing methods also overlook the performance impact of cache contention during parallel execution. In Gustavson's algorithm, each thread maintains an exclusive row-level accumulator whose size scales with the nonzero density of its assigned rows. Reading Figure 3 row-wise, each row of A gathers the B rows indexed by its nonzeros into that accumulator, so cf measures the nonzeros of B streamed per accumulator entry; a high-cf row thus places significantly greater pressure on shared caches than a low-cf row. As high-cf partitions generally contain more high-cf rows and Gustavson's algorithm is typically parallelized row-wise, they are more prone to inter-thread cache interference than low-cf partitions. Figure 5(a) shows this effect using the same skewed matrix as in Figure 4. When all threads compute their assigned default partitions, the runtime of a high-cf partition

[7] We use the term "clustering" to describe this effect, following the notion of "primary clustering" in open-addressing hashing.

[8] The compression factor (cf) is defined as the number of flop counts divided by the nnz of the result matrix C, i.e., $cf = \frac{flop}{nnz(C)}$ [45].

[9] Only multiplications are counted, as the number of additions is identical.

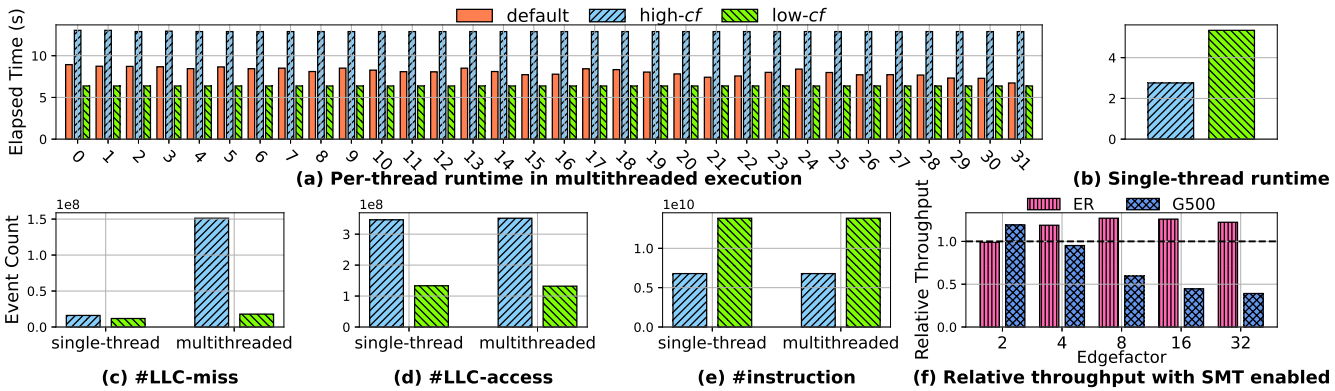


Figure 5: (a) Per-thread computation time of HASH-based SpGEMM on a G500 skewed matrix (scale=19, *edgefactor*=8) under three settings: all threads execute their default partitions; all execute a high-cf partition (thread-0's); all execute a low-cf partition (thread-31's). (b) Computation time of the low-cf and high-cf partitions under single-threaded execution. (c)–(e) LLC misses, LLC accesses, and instruction counts for the low-cf and high-cf partitions under both single-threaded and multithreaded execution. (f) Normalized throughput of HASH-based SpGEMM with SMT enabled on ER and G500 matrices (scale=18) across varying *edgefactor* values. The black dashed line marks the relative throughput without SMT.

(originally assigned to thread-0 in the flop-count-based scheme) increases substantially: from 2.76 seconds in single-threaded execution (cf. Figure 5(b)) to 8.93 seconds under default multithreaded partitioning, and further to 13.01 seconds if all threads execute this high-cf partition. Note that no sharing benefit arises in this case, as each thread operates on its own exclusive accumulator; instead, these accumulators compete for the same shared cache, amplifying interference as more threads process high-cf regions. In contrast, a low-cf partition (originally assigned to thread-31) sees only a modest runtime increase, from 5.36 to 6.73 seconds.

Further profiling reveals that cache contention is the root cause of this disparity: Figures 5(c)–(e) show a sharp rise in last-level cache (LLC) misses for the high-cf region, while instruction count and LLC accesses remain relatively stable, confirming cache contention as the primary bottleneck [10] .

The problem is further exacerbated under simultaneous multithreading (SMT): Figure 5(f) shows that non-skewed workloads (e.g., ER matrices with typically low cf) benefit from SMT, whereas skewed workloads (e.g., G500 matrices with high-cf subregions) suffer increasing throughput degradation as density grows, since higher density amplifies the skew and thus intensifies SMT-induced contention. This underscores the importance of contention-aware scheduling; neglecting this factor not only compromises load balance but also degrades overall SpGEMM efficiency.

Ignoring multi-dimensional skew and overlooking cache contention together compound into a macro-level penalty on system-wide bandwidth utilization. As Figure 6 illustrates on a skewed workload, bandwidth resources are only fully utilized during the initial burst of parallel execution; as straggler cores emerge, system-wide bandwidth consumption starts to drop, leaving expensive memory resources idle. This renders prevailing bandwidth expansion technologies, such as CXL, far less effective, as the slowest thread governs total runtime, and its prolonged latency [94] further widens the gap despite the increased aggregate bandwidth available.

[10]Write-out time is excluded to directly address computation-level contention.

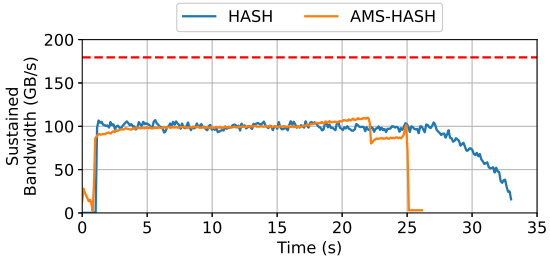
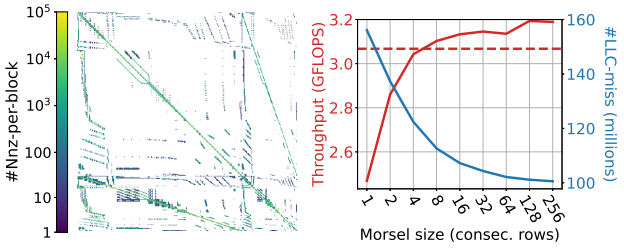


Figure 6: Comparison of sustained bandwidth of the numeric phase between flop-count-based hashing-based SpGEMM and the AMS variant on a G500 skewed workload (scale=19, *edgefactor*=16); the horizontal red dashed line denotes system memory bandwidth (179.50 GB/s).

In contrast, a contention-aware load-balancing approach (our AMS method) sustains markedly higher bandwidth execution.

2.3 Revisiting Dynamic Scheduling

Having established that both multi-dimensional skew and cache contention undermine existing scheduling strategies, we now revisit the scheduling paradigm that enables our solution. Specifically, we revisit *dynamic scheduling* [76, 79]: rows of A are placed into a shared task pool, and threads repeatedly fetch and process one row at a time, achieving nearly 100% load balance. While simple and theoretically well-balanced, this approach has been shown to underperform compared to flop-count-based scheduling and has largely been dismissed in recent literature, which attributes the performance penalty primarily to the synchronization overhead associated with dynamic task dispatch [45, 76]. However, we contend that it is task granularity, rather than synchronization overhead, that plays the dominant role in determining performance. When task size is carefully managed, dynamic scheduling can match, if not outperform, flop-count-based schemes.



(a) NNZ-pattern (b) Throughput & LLC-miss

Figure 7: (a) Nonzero pattern of the `vas_stokes_1M` matrix. (b) Throughput and LLC misses of hashing-based SpGEMM on `vas_stokes_1M` using dynamic scheduling with varying morsel sizes (measured in consecutive rows per morsel); the horizontal red dashed line indicates the throughput of the flop-count-based scheme.

Take a real-world SpGEMM workload, `vas_stokes_1M` [1, 31], for example. The matrix exhibits clear spatial locality across rows, particularly within blocks of consecutive rows (Figure 7(a)). We benchmark the performance of hashing-based SpGEMM using dynamic scheduling with varying task granularities (i.e., the number of consecutive rows per task). Figure 7(b) shows that throughput improves with increasing granularity, plateauing around 128 rows, and surpasses its flop-count-based counterpart due to better load balance. Further analysis of LLC miss counts reveals that larger task sizes result in fewer misses, suggesting better preservation of spatial locality. It is worth noting that many real-world matrices exhibit strong spatial locality in their nonzero patterns, and prior works have introduced several offline reordering algorithms to enhance locality for downstream operations [5, 25, 100]. Based on these observations, we maintain that dynamic scheduling remains a practical and efficient strategy as long as spatial locality is effectively preserved. We therefore build on this insight for the design of our AMS approach.

3 METHODOLOGY

This section presents our proposed framework, adaptive morsel scheduling (AMS). We begin with the core concept of morsel-wise dynamic scheduling, our strategy for addressing multi-dimensional skew and achieving efficient load balancing in SpGEMM. Building upon this foundation, we present two key techniques, contention-aware load balancing (CALB) and data-conscious simultaneous multithreading activator (SMTact), designed specifically to alleviate on-chip cache contention. Furthermore, we explain how the morsel-wise framework enables our hybrid accumulation mechanism (HYB) approach for optimizing SpGEMM execution. Finally, we discuss the broader implications of AMS: how the interplay between load balance and cache efficiency governs the benefit of bandwidth-expanded memory, how the underlying design principle carries over to relational and graph processing, and how AMS extends to other sparse matrix multiplication kernels.

We note that AMS targets the numeric phase of SpGEMM, which dominates both computation and memory traffic. While the same ideas can be applied to the symbolic phase, the benefits are less pronounced; for simplicity, we use standard dynamic scheduling with fixed-size morsels for that phase. We defer a detailed discussion of the symbolic phase, together with a breakdown of its cost, to Appendix A.

Before detailing our techniques, we briefly digress to outline three key design principles that an efficient SpGEMM solution should satisfy, all of which are met by our proposed AMS.

(1) **Simple and lightweight.** SpGEMM has been extensively optimized over decades [38], and any added complexity can degrade its performance. This is especially critical when SpGEMM is repeatedly invoked as a core kernel in real-world systems [21, 56, 69, 76]. Thus, any proposed method must be simple and lightweight to avoid introducing significant execution penalties.
(2) **Non-disruptive to non-skewed workloads.** While skewed patterns are common in real-world workloads, non-skewed distributions (e.g., uniform distribution, symmetric distribution) also arise frequently. A design that benefits skewed cases but penalizes non-skewed ones forces practitioners to make case-dependent trade-offs, increasing system complexity and operational overhead.
(3) **Generalizable across algorithms and platforms.** An effective SpGEMM solution should generalize across algorithmic variants and hardware platforms. In practice, SpGEMM is used in diverse contexts with varying requirements, e.g., numerical computing often demands sorted output [30], while relational and graph queries typically accept unsorted results [5, 25, 43]. Also, it should adapt to different hardware architectures, where sensitivities to compute and memory bottlenecks vary. A practical solution must therefore accommodate both, supporting sorted and unsorted pipelines while maintaining robust performance across platforms.

3.1 Morsel-Wise Dynamic Scheduling

We previously demonstrated that, by using morsels composed of a sufficient number of consecutive rows, dynamic scheduling is able to effectively preserve spatial locality in sparse matrices. This method significantly reduces LLC misses with negligible synchronization overhead (§ 2.3). We therefore adopt this line of design for SpGEMM execution.

While scheduling work in morsels of a fixed row count helps reduce cache thrashing, we argue that this approach leaves room for further improvement, as it does not address the *multi-dimensional skew* identified in § 2.2. Specifically, it has two limitations: (1) Fixed-row-count morsels do not account for differences in flop count or result *nnz*, making it difficult to balance computation and materialization costs. This limitation is particularly pronounced in skewed workloads, where high-cf submatrices incur high computational expense while others involve substantial write-out overhead. Due to such disparities in two dimensions, modeling inter-thread cache contention becomes challenging. (2) This fixed-row strategy also fails to control the size of the resulting output. A single morsel containing a few dense rows, particularly in skewed workloads, can produce more data than the CPU cache can accommodate, making it challenging to construct a cache-aware analysis model [48, 74, 86].

To address these issues, we propose scheduling morsels based on a fixed result size, i.e., a target $nnz(C)$ per morsel. Recall that SpGEMM consists of two phases, where the symbolic phase determines the per-row nonzero structure. We thus leverage this information to form morsels of consecutive rows in an adaptive

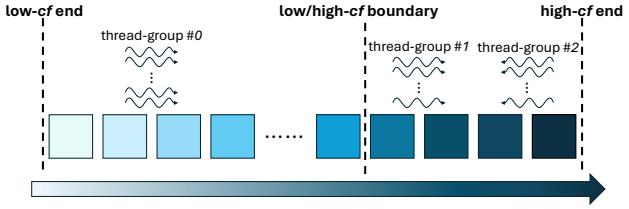


Figure 8: Thread scheduling with contention-aware load balancing (CALB). Darker colors indicate higher compression factors (cf).

manner, such that each morsel contains approximately the same number of output nonzeros. By fixing the materialization cost per morsel, this approach normalizes one dimension of skew, allowing us to focus on the remaining computational cost dimension. It also decouples cf from the output size that flop-count-based partitioning leaves uncontrolled: with $nnz(C)$ fixed, cf varies solely with the flop count.

Additionally, to facilitate on-chip cache modeling, we limit the morsel size to half of the L2 cache capacity [11].

We stress that this size is a *soft target* rather than a hard bound, and that it budgets only the output footprint $nnz(C)$ of a morsel, not the accumulator or the rows of B streamed into it. AMS appends whole consecutive rows to a morsel until the target is reached, and because an output row is atomic in the row-parallel Gustavson formulation, AMS never splits a row across threads. A row whose own output already exceeds the target therefore forms a morsel by itself and may be far larger than the target. Such oversized rows are common rather than exceptional in skewed workloads, and we show in Appendix B that they do not degrade AMS.

This strategy is simple and efficient, and it forms the foundation for our subsequent designs. More importantly, it introduces no additional algorithmic phase, fully aligning with our design principle of being **simple and lightweight**.

3.2 Contention-Aware Load Balancing

We now elaborate on how we alleviate inter-thread on-chip cache contention, which can degrade SpGEMM performance even under balanced workload distribution. Since we form morsels based on a fixed result size, we can easily compute the compression factor (cf) for each morsel. Recall from § 2.2 that contention originates at the row level: a high-cf row streams more nonzeros of B per accumulator entry (see Figure 3), and is thus contention-prone regardless of how rows are grouped. With $nnz(C)$ fixed per morsel, a high-cf morsel likewise carries more computation against an identical footprint, and remains more prone to contention than a low-cf one. Therefore, we aim to co-execute high-cf and low-cf morsels concurrently, alleviating cache contention and reducing susceptibility to inter-thread interference.

To that end, we first sort all morsels in ascending order of cf and empirically set a boundary between low-cf and high-cf regions, typically between the 80th and 95th percentile. This is motivated by the observation that most skewed workloads follow a power-law distribution [5, 25, 100, 113], where a small fraction of dense rows contribute to the majority of the workload. We then divide the available threads into three groups: thread group #0 starts from the low-cf end and progresses toward the high-cf end; group #1 begins at the low/high-cf boundary and proceeds toward the high-cf end; group #2 starts at the high-cf end and moves backward toward the boundary (see Figure 8). The majority of threads are allocated to group #0 to handle low-cf tasks, while smaller subsets are assigned to groups #1 and #2 for high-cf regions. This arrangement ensures that only a limited number of high-cf morsels are processed concurrently. Dividing the remaining threads into groups #1 and #2 further helps mitigate cache contention by avoiding simultaneous execution of multiple contention-prone morsels at the high-cf end. Meanwhile, co-running low-cf morsels, which are less sensitive to cache interference, alongside high-cf morsels helps absorb contention effects and improves overall execution efficiency.

While this scheme alleviates cache contention, it does not guarantee load balance. Some thread groups may finish early and become idle, leading to underutilization of computational and memory bandwidth resources. To address this, we allow early-finished threads to assist other active groups. Specifically, if group #0 reaches the boundary early, it merges with group #1 to continue processing toward the high-cf end. Likewise, early-finished threads from groups #1 and #2 join group #0 to help complete tasks in the low-cf region. This cooperative mechanism ensures that all threads remain active, ensuring efficient use of computation and memory bandwidth.

The CALB scheme is conceptually simple, yet novel and highly effective. To the best of our knowledge, it is the first SpGEMM load-balancing technique that explicitly considers on-chip cache contention as a scheduling factor and thereby delivers tangible performance benefits. Moreover, the method is platform-agnostic and can be seamlessly integrated into any SpGEMM algorithm based on Gustavson's formulation, adhering to the design principle of being **generalizable across algorithms and platforms**.

3.3 Data-Conscious Simultaneous Multithreading Activator

While SpGEMM is generally considered bandwidth-bound, its computational phase still incurs a non-negligible overhead. This overhead can typically be mitigated by leveraging simultaneous multithreading (SMT), which increases the logical core count to effectively improve computational throughput. However, as shown in Figure 5(f), this performance benefit from SMT is primarily observed in non-skewed workloads; in contrast, skewed workloads degrade significantly when SMT is enabled.

We attribute this disparity to the increased on-chip cache pressure from SMT. Although SMT increases the logical thread count, it does not expand the physical cache. Consequently, high-cf morsels suffer from intensified cache contention, an issue that is particularly exacerbated in skewed workloads where cf values are disproportionately large for a few morsels. These specific morsels are highly vulnerable to SMT-induced interference, which in turn leads to substantial performance degradation.

[11] We empirically find that setting the morsel size to $\frac{1}{8}$ and $\frac{1}{2}$ of the L2 cache offers the best trade-off between performance and cache contention.

To mitigate this issue while preserving the computational benefits of SMT for non-skewed workloads, we adopt a selective activation strategy of SMT, namely data-conscious simultaneous multithreading activator (SMTact). Specifically, we set a threshold based on the highest cf observed. SMT is disabled if the workload contains any morsels exceeding this threshold, and enabled otherwise. This strategy ensures that SMT is applied only when the workload is less prone to cache contention [12]. The approach is simple yet effective, as it balances the computational benefits of SMT against the risk of elevated cache contention, aligning with our second design principle of being **non-disruptive to non-skewed workloads**.

3.4 Hybrid Accumulation Mechanism

Due to the inherently sparse and irregular nature of SpGEMM, computing a single output row generates multiple intermediate sparse products. These products must then be merged, a process that requires appropriate accumulators to mitigate two challenges: severe irregular memory access patterns and the high cost of managing collisions for identical column indices. Recognizing these challenges, numerous works have focused on developing efficient accumulators [30, 42, 76]. However, the effectiveness of any given accumulation strategy varies dramatically based on the characteristics of the underlying matrix morsel. This makes a static, one-size-fits-all accumulator inherently suboptimal.

To address this, we leverage the morsel-wise framework of AMS to introduce a hybrid accumulation mechanism (HYB) that dynamically selects the most suitable accumulator for each morsel. Specifically, we make this choice based on the aforementioned morsel-wise cf value, selectively choosing between two widely used accumulators: hash tables (HASH) [75, 76], which use linear probing and multiplicative hashing to reduce collisions, and expand-sort-compress (ESC) [30, 45], which relies on highly optimized radix sorting to alleviate common logarithmic sorting overheads.

The choice between these two accumulators is highly dependent on the morsel-wise cf. As ESC generates sorted output, it executes more instructions than HASH. However, this instruction overhead is not necessarily detrimental, particularly for low-cf morsels. For such morsels, the number of intermediate products is similar to the final number of nonzeros, leading to a moderate cost for merging and sorting. In contrast, HASH becomes more populated for low-cf morsels, and the overhead of hash computation and probing becomes more pronounced. For high-cf morsels, however, HASH is the better choice, as a massive number of intermediates are hashed into a few slots, greatly reducing potential cache misses. Conversely, ESC must pay a significant cache miss cost to merge a large volume of intermediate results into a small final output. Figure 9 corroborates this by microbenchmarking the LLC misses of HASH and ESC, which shows that ESC suffers significantly as the cf increases (e.g., in skewed G500 workloads) compared to non-skewed (ER) workloads.

Therefore, we empirically identify a cf threshold, using ESC for morsels with a cf below this value and opting for HASH for morsels above it [13]. This morsel-wise selection also offers an architectural benefit: selecting the accumulator on a per-morsel basis alleviates the instruction-level branch-misprediction costs that finer-grained (per-row) selection would incur.

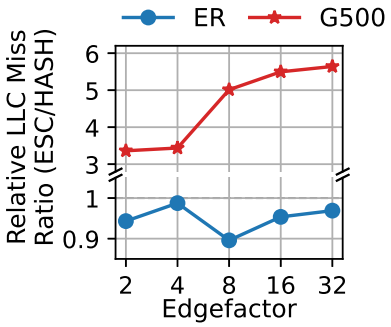


Figure 9: The LLC miss ratio of ESC normalized to the HASH (i.e., LLC Misses of ESC / LLC Misses of HASH), evaluated on ER and G500 datasets at Scale 18 with varying edgefactors (higher edgefactors suggest denser workloads).

It is worth noting that by including HASH, our HYB approach may produce an unsorted result matrix, which is acceptable for applications like relational queries and graph analytics that do not require sorted output [54, 75]. However, if sorted output is necessary, HYB can be easily adapted: instead of choosing between HASH and ESC, it can be configured to choose the efficient strategy from a set of sorted accumulators, such as ESC, HEAP [76], SPA [42], or even a hash-then-sort approach [27], to ensure a fully sorted output. Consequently, this adaptive approach ensures each morsel is processed with the most suitable accumulator, leading to better overall SpGEMM performance regardless of the output ordering requirement.

3.5 Implications and Broader Applicability

Having presented AMS's components, we now step back to consider what they imply beyond the SpGEMM kernel itself. AMS highlights an important interaction between load balance and cache efficiency. Static partitioning preserves locality but leaves cores idle under skew, thereby underutilizing memory bandwidth; dynamic scheduling keeps cores busy but raises cache contention once the working sets of co-executing morsels exceed the shared cache. Effective scheduling must therefore maintain enough concurrency to saturate the memory system while limiting avoidable cache traffic, which is what AMS achieves by scheduling morsels from cheap metadata. This interaction is precisely what makes bandwidth-expanded memory [105] pay off, since additional bandwidth is realized only when enough cores issue concurrent requests, and it is why combining balanced execution with reduced interference lets AMS sustain throughput beyond the local-DRAM peak (§ 4.4).

The same design principle can apply beyond SpGEMM when lightweight metadata is available to estimate each morsel's processing cost and demand on the shared cache. For SpGEMM, the symbolic phase provides this information directly. Other operators can obtain approximate metadata through a preliminary statistics pass [9, 26] or sampling [102, 104]; exact estimates are unnecessary because CALB uses only the relative ordering of morsels by cache

[12] We also experiment with using quantiles of the cf distribution instead of the maximum, and observe comparable performance with an appropriately chosen threshold.
[13] We find that a cf between 1.5-1.7 provides a robust threshold for selecting the appropriate accumulator.

demand and SMTact only tests whether a threshold is crossed. This creates opportunities to apply contention-aware morsel scheduling to other operators over relational and graph data with skewed, cache-resident state [4, 32, 43, 54, 93]. For example, a morsel-driven engine [34, 66] could use partition-size estimates to approximate cache demand when scheduling hash-aggregation or hash-join tasks over skewed keys.

A direct and notable extension of AMS, beyond SpGEMM and the relational query and graph operators it underpins, is to apply it atop other sparse matrix multiplication kernels, e.g., sparse matrix-matrix multiplication (SpMM) and sparse matrix-vector multiplication (SpMV), which multiply a sparse matrix by a dense operand. Their output is dense and of fixed size, which renders HYB unnecessary and makes morsel formation by a target result size meaningless; we instead group consecutive rows by a target *nnz* of the left-hand operand. Although the skew thereby reduces to a single dimension (flop count), load imbalance and cache contention persist, as morsels of many short rows and morsels of few long rows differ markedly in data reuse [39, 108], so CALB and SMTact apply as they do in SpGEMM [68]. Since most SpMM and SpMV solutions likewise follow a Gustavson-style row-wise formulation, the adaptation is straightforward, and we report the resulting speedups in § 4.5.

4 EXPERIMENTAL EVALUATION

This section evaluates AMS in addressing multi-dimensional skew and on-chip cache contention. We begin with the experimental setup (§ 4.1), then benchmark AMS on SpGEMM across diverse workloads (§ 4.2) and dissect CALB and SMTact to validate their contention alleviation (§ 4.3). We further evaluate AMS on a bandwidth-expanded CXL system to show improved bandwidth utilization (§ 4.4). We then demonstrate its broad applicability on other sparse matrix multiplication kernels (§ 4.5), on relational join-aggregate and join-project queries (§ 4.6), and on graph-analytic workloads (§ 4.7), including PathSim meta-path construction [93], breadth-first search, triangle counting, and an end-to-end Markov clustering pipeline (HipMCL). We close with a discussion (§ 4.8) that distills the key findings across these workloads and platforms, and relates them back to the design rationale of AMS.

4.1 Experimental Setup

We conduct experiments on a single-socket machine running Linux kernel 6.11.0. This system is equipped with an Intel Xeon® Gold 6430 CPU, which features 32 physical cores and supports 64 logical threads via simultaneous multithreading (SMT) [14] . Each core includes a 48 KB L1 data cache, a 32 KB L1 instruction cache, and a 2 MB L2 cache, while all cores share a 60 MB last-level cache (LLC). The system comprises 192 GB of DDR5 DRAM, providing 179.50 GB/s of memory bandwidth. We also conduct experiments on a CXL-augmented memory system to validate the generalizability of AMS, with the setup details deferred to § 4.4.

Following prior settings [6, 19, 21, 22, 75], we conduct our primary performance analysis using R-MAT matrix squaring workloads [23, 40, 59]. We employ three representative sets of R-MAT matrices to model different workload characteristics: ER ($a = b = c = d = 0.25$), SSCA ($a = 0.60, b = c = 0.13, d = 0.14$), and G500 ($a = 0.57, b = c = 0.19, d = 0.05$), which represent non-skewed, skewed, and highly skewed workloads, respectively. All R-MAT generators support control over size and density via two parameters: *scale* (determining the number of rows, i.e., 2^{scale}) and *edgefactor* (defining the average nonzeros per row). We sweep *scale* over 19–21 for ER, 18–20 for SSCA, and 17–19 for G500, with *edgefactor* over 2–32 in all three. For G500 workloads, higher *edgefactors* amplify multi-dimensional skew and cache contention (§ 2.2). Additionally, we evaluate SpGEMM performance using real-world workloads from the SuiteSparse Matrix Benchmark [31] to validate the robustness of AMS. We perform matrix squaring on 119 SuiteSparse matrices, with row counts ranging from 100K to 10M and nonzero

[14]We disable Intel Speed Select Technology - Base Frequency to ensure equal performance for all physical cores [99].

Table 1: Summary of all evaluated SpGEMM methods

Method	Category	Sorted	Description
Algorithmic baselines (built upon by our AMS methods)			
`HASH [76]`	Baseline	✗	Hashtable accumulator with flop-count-based scheduler.
`ESC [30]`	Baseline	✓	Radix-sort (expand–sort–compress) accumulator with flop-count-based scheduler.
External / industry competitors			
`HEAP [76]`	Competitor	✓	Heap accumulator with flop-count-based scheduler.
`PB [45]`	Competitor	✓	Outer-product SpGEMM flow with flop-count-based scheduler.
`MKL [56]`	Competitor	✗	Intel oneMKL SpGEMM — non-sorted variant.
`MKLS [56]`	Competitor	✓	Intel oneMKL SpGEMM — sorted variant.
Proposed AMS-based methods			
`AMS-HASH`	Proposed	✗	AMS approach using a HASH accumulator.
`AMS-ESC`	Proposed	✓	AMS approach using an ESC accumulator.
`AMS-HYB`	Proposed	✗	AMS with per-morsel adaptive HASH/ESC selection.

Table 2: Summary of 20 representative matrices evaluated.

Matrix	row(A)	nnz(A)	gini(A)[†]	flop[‡]	nnz(A^2)	cf
hugetric-00020	7.13M	21.37M	0.0003	64.08M	44.61M	1.4363
GL7d18	1.96M	35.60M	0.0395	890.29M	843.47M	1.0555
333SP	3.72M	22.22M	0.0559	134.79M	71.97M	1.8730
NLR	4.17M	24.98M	0.0736	152.77M	82.02M	1.8626
M6	3.51M	21.01M	0.0742	128.49M	68.99M	1.8625
AS365	3.80M	22.74M	0.0765	138.91M	74.62M	1.8616
auto	448.70K	6.63M	0.1016	101.25M	33.06M	3.0634
delaunay_n23	8.39M	50.34M	0.1220	316.97M	173.67M	1.8251
nv2	1.46M	37.48M	0.1401	1.28B	292.14M	4.3475
Freescale1	3.43M	17.06M	0.1858	90.33M	38.31M	2.3580
gsm_106857	589.45K	21.76M	0.2240	947.26M	132.40M	7.1546
amazon-2008	735.33K	10.32M	0.2856	218.17M	63.16M	3.4546
webbase-1M	1.01M	3.11M	0.3455	1.37B	1.31B	1.0475
vas_stokes_2M	2.15M	65.13M	0.3771	6.95B	1.17B	5.9452
rel9	9.89M	23.67M	0.4016	2.10B	2.04B	1.0300
vas_stokes_1M	1.10M	34.77M	0.4081	4.39B	721.63M	6.0776
patents	3.78M	14.98M	0.6585	178.30M	132.12M	1.3496
cit-Patents	3.78M	16.52M	0.6844	239.46M	162.35M	1.4749
NotreDame_www	325.73K	929.85K	0.7825	418.01M	288.98M	1.4465
web-NotreDame	325.73K	1.50M	0.8523	503.57M	293.20M	1.7175

[†] Gini coefficient [63] of the row-wise nonzeros in A.
[‡] Only multiplications are counted, as additions are equal in number.

counts from 146K to 316M. This range is deliberate: even the smallest matrix in it does not fit in the 2 MB private L2 cache [15], so that every workload genuinely exercises the cache hierarchy under study rather than residing in a private cache, while the largest ones still complete on our platform without triggering out-of-memory errors. For a clear presentation, we present results for 20 representative matrices (Table 2), ordered by their Gini coefficient [16], as other matrices show similar trends. Beyond SpGEMM, we evaluate AMS on relational join-aggregate and join-project queries, on graph-analytic workloads such as PathSim meta-path construction [93], and on other sparse matrix multiplication kernels, with setups and workloads detailed in § 4.6, § 4.7, and § 4.5, respectively.

We integrate our AMS framework into two prevalent Gustavson-based SpGEMM algorithms: the non-sorted HASH algorithm [75, 76] and the sorted ESC algorithm [14, 30]. These are referred to as AMS-HASH and AMS-ESC, respectively [17] . Moreover, we implement our hybrid accumulation mechanism (HYB) as AMS-HYB, which dynamically selects between HASH and ESC accumulators. We empirically tune the parameters for AMS-based approaches, configuring morsel size at half the L2 cache size, and distributing threads using a 60%-25%-15% ratio across groups #0, #1, and #2, respectively, for optimal CALB performance.

We compare our methods against several state-of-the-art baselines (see Table 1 for an overview). These include: (1) the same HASH and ESC algorithms driven by a state-of-the-art flop-count-based scheduler; (2) the sorted HEAP algorithm [75, 76], which exploits the implicit column ordering of input matrices to reduce sorting overhead at the cost of increased memory usage; (3) the outer-product-based PB method [45]; and (4) the de facto industry solution, Intel MKL [56], including both its sorted (MKLS) and non-sorted (MKL) variants. While other SpGEMM algorithms, such as SPA [42] and SA [48], were also evaluated, we omit their results for conciseness as they consistently underperformed.

All implementations are compiled using `GCC-11.5.0` with the `-O3` optimization flag and parallelized with OpenMP [29]. Baselines are manually tuned for optimal performance, which includes enabling SMT for non-skewed workloads and disabling it for skewed ones. Our AMS-based methods are tuned with equal care, sweeping the thread group sizes, the low/high-cf boundary, and the SMTact threshold. As in prior works, throughput, measured in GFLOPS (giga floating-point operations per second), serves as the primary evaluation metric. Unless a measurement is explicitly labeled as numeric-phase profiling, all reported runtimes and throughputs cover the complete SpGEMM pipeline, i.e., symbolic execution, prefix sum and output allocation, AMS preparation, numeric execution, and output finalization. All reported results are averaged over 10 runs [18].

4.2 SpGEMM Performance Evaluation

Figure 10 presents the SpGEMM performance on R-MAT and SuiteSparse workloads, and shows that our AMS-based algorithms consistently achieve high performance across this broad spectrum. Specifically, the proposed AMS-HYB is generally the superior solution: it maintains performance comparable to AMS-HASH in skewed workloads (SSCA and G500) while remaining as competitive as AMS-ESC in non-skewed, high-*edgefactor* workloads (ER). This outcome validates both the effectiveness of our HYB design and the soundness of our cf-based criterion for adaptively selecting the best morsel-wise accumulator for efficient multiplication.

Beyond the superior performance of AMS-HYB, the AMS framework itself significantly boosts both its non-sorted (AMS-HASH) and sorted (AMS-ESC) variants. These methods perform comparably to their respective baselines on ER matrices but deliver a substantial performance advantage on the skewed SSCA and G500 matrices. The advantage moreover tracks the degree of skew for the non-sorted variant: at matched *scale* and *edgefactor*, where SSCA and G500 differ only in their R-MAT probabilities, AMS-HASH gains more on the more skewed G500 side in every such pair, reaching 1.44× against 1.23× at scale 19 and *edgefactor* 2 (Appendix B). This demonstrates that AMS's dynamic scheduling, when paired with appropriately sized morsels, effectively addresses multi-dimensional skew on skewed workloads without sacrificing performance on non-skewed ones. The strong results on ER matrices (Figure 10(a)–(c)) also validate the effectiveness of SMTact, which selectively enables SMT in low-contention scenarios to enhance computational throughput, a critical factor in non-skewed workloads.

Comparing the impact of AMS across different algorithms, we observe that the performance improvement is more pronounced in HASH than in ESC. This is because ESC relies on radix sort, which has a complexity of $O(\omega \cdot n)$ and correlates directly with the flop count. Although HASH-based methods also exhibit $O(n)$ complexity in hash table lookups, their efficiency is more susceptible to "clustering" effects, making flop-count-based models less effective in accurately estimating cost and balancing workloads. In contrast, our AMS approach is agnostic to the underlying SpGEMM algorithm. For both non-sorted and sorted algorithms, AMS achieves performance gains of up to 81.01% and 36.03%, respectively, demonstrating its general applicability across algorithmic variants.

To further illustrate the source of these gains, Figure 11 drills down into the per-thread elapsed time, comparing AMS-based methods against their respective baselines on a representative skewed workload. The baseline HASH and ESC exhibit significant thread-level runtime variance, confirming that flop-count-based scheduling fails to balance the actual workload under skew. In contrast, AMS-based variants achieve substantially more uniform per-thread runtimes, demonstrating that our scheduling framework effectively

[15] Storing a matrix of n rows and nnz nonzeros in CSR with 8-byte indices and double-precision values takes $8(n+1) + 16\,nnz$ bytes, so the lower end of this range already occupies about 3 MB, whereas the upper end occupies about 5.1 GB.

[16] The Gini coefficient measures inequality in the row-wise nonzero distribution; a value of 0 indicates uniform distribution, while values approaching 1 indicate extreme skew. Coefficients above 0.30 generally indicate skewed workloads [31, 63].

[17] Sorted algorithms generally incur more overhead to produce row-wise sorted output.

[18] The source code, data, and/or other artifacts have been made available at https://github.com/fukien/casmm.

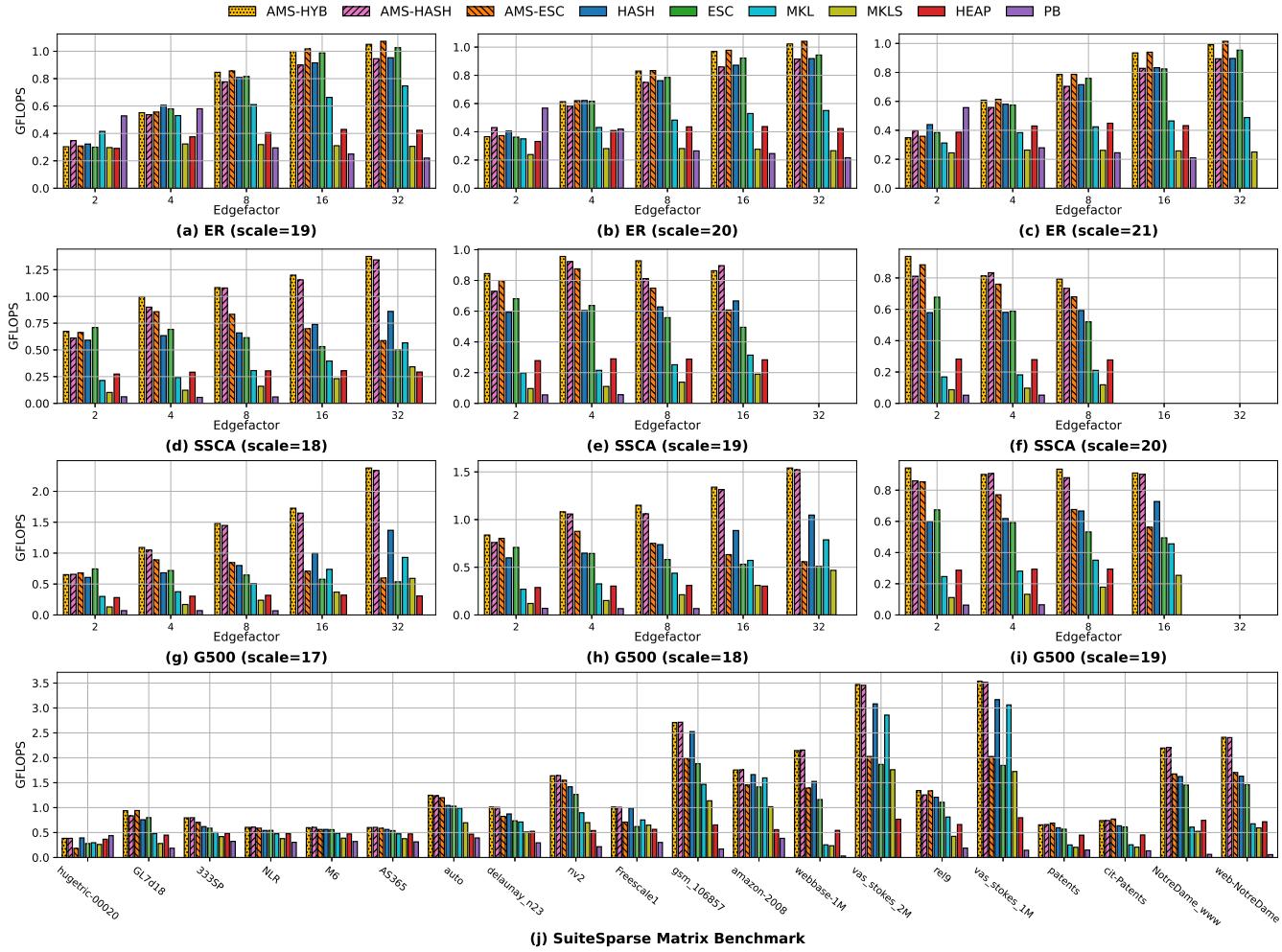


Figure 10: SpGEMM performance evaluation. Subfigures (a)–(i) present results for R-MAT matrices (ER, SSCA, G500) across varying scales and *edgefactor* values, representing changes in matrix dimension and density, respectively. Subfigure (j) displays results for 20 representative matrices from the SuiteSparse collection, ordered on the x-axis by their Gini coefficient (ascending), which correlates with increased workload skew. Empty bars signify out-of-memory (OOM) failures.

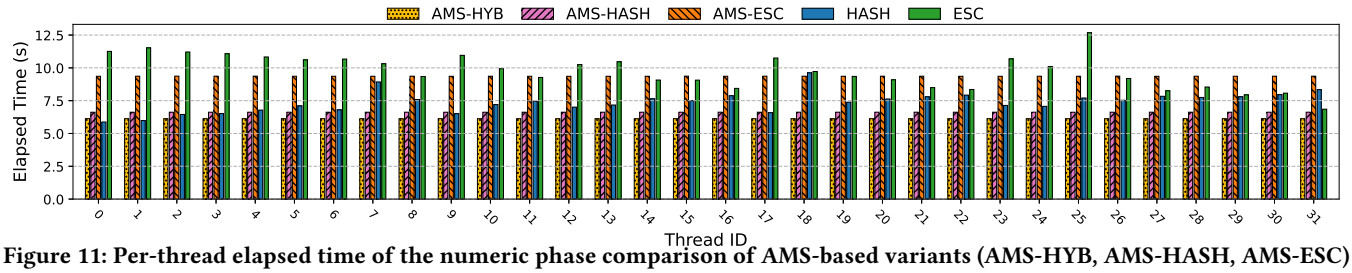


Figure 11: Per-thread elapsed time of the numeric phase comparison of AMS-based variants (AMS-HYB, AMS-HASH, AMS-ESC) and their baseline counterparts (HASH, ESC) on a G500 skewed matrix (scale=19, *edgefactor*=8).

addresses the load imbalance problem inherent in skewed workloads.

Despite these advantages, Figure 10 shows that AMS-based algorithms may occasionally underperform their flop-count-based counterparts in highly sparse cases (e.g., G500 with scale 17 and *edgefactor* 2), primarily due to overhead from cf-based morsel

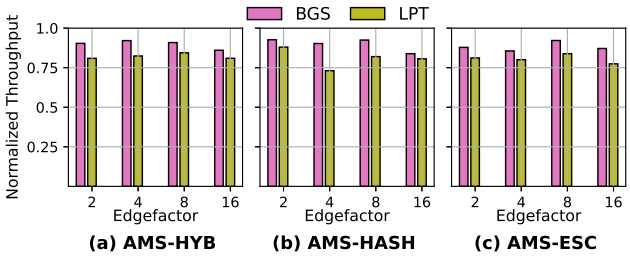


Figure 12: Throughput comparison of BGS and LPT scheduling strategies against CALB. The y-axis shows throughput on G500 (scale=19) workloads, normalized to CALB.

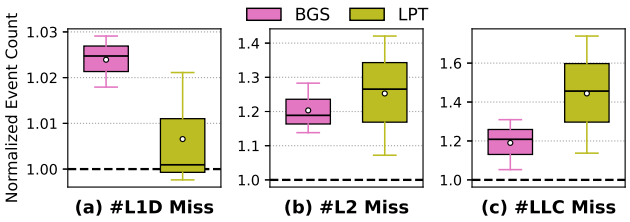


Figure 13: Distribution of the L1D, L2, and LLC misses incurred by the BGS and LPT scheduling strategies on G500 workloads (scale=19), each normalized to the corresponding CALB-based execution (dashed line at 1.0). Every box aggregates the samples of all three AMS variants, i.e., AMS-HYB, AMS-HASH, and AMS-ESC. Within each box, the solid line marks the median, the white circle the mean, and the whiskers extend to the most extreme samples within 1.5× the interquartile range. (a) L1D misses. (b) L2 misses. (c) LLC misses.

sorting. Nevertheless, this penalty diminishes as matrix density increases, underscoring the lightweight nature of AMS in addressing load imbalance.

Turning to the other methods, the baseline HASH and ESC algorithms generally trail their AMS-enhanced counterparts, followed by MKL/MKLS. In contrast, HEAP and PB consistently show inferior performance with more unfinished runs, owing to their notably larger memory demands. An exception is PB's performance on highly sparse non-skewed workloads (ER matrices, *edgefactor* 2), due to its outer-product-based formulation that maximizes data reuse [45]. However, this performance advantage quickly diminishes at higher *edgefactor* values, implying limited applicability across diverse workloads. These observations further underscore the need for adaptive SpGEMM processing, as no single formulation dominates across the density spectrum.

We next compare AMS against the de facto industry solution, Intel MKL. On R-MAT workloads, AMS-HYB surpasses the non-sorted MKL by up to 3.83× and the sorted MKLS by up to 8.45×, while AMS-ESC delivers up to 7.66× over MKLS.

Moreover, the analysis of SuiteSparse matrices (Figure 10(j)) substantiates these performance advantages on real-world workloads, confirming AMS's broad practical applicability. The advantage over MKL persists there: AMS-HYB leads both MKL and MKLS on all 20 matrices, while AMS-HASH and AMS-ESC outperform their flop-count-based counterparts on 19 of the 20 matrices, by up to 47.46% and 25.92%, respectively, confirming that AMS generalizes to real-world sparsity patterns.

4.3 Dissecting AMS: CALB and SMTact

4.3.1 Effectiveness of CALB. We now examine the performance benefits of our CALB approach. Designed to alleviate cache contention while maintaining load balance, CALB is evaluated against two alternative scheduling techniques also applicable to our cf-sorted morsels: (1) a bidirectional greedy scheduling scheme (BGS), where threads split into two equal-sized groups processing morsels from opposite ends toward the center, and (2) the classical longest-processing-time-first strategy (LPT) [44], in which all threads start from the high-cf end and progress toward the low-cf end.

Figure 12 reports the throughput of BGS and LPT, normalized to our CALB method, for three AMS-based algorithms on G500 workloads (scale=19). The results show that CALB consistently outperforms both alternatives, achieving 7.90%–19.26% and 8.53%–36.80% higher throughput than BGS and LPT, respectively. While BGS also aims to mitigate contention by avoiding excessive thread concentration on high-cf regions, our three-group division strategy in CALB proves more effective at reducing cache pressure, especially at the high-cf end. In contrast, LPT performs poorly precisely because it schedules all threads to handle the most demanding, high-contention tasks simultaneously, thereby exacerbating on-chip cache contention.

This explanation rests on cache contention, so we verify it by direct measurement rather than by inference from throughput. Figure 13 profiles the L1D, L2, and LLC misses incurred by BGS and LPT with hardware performance counters, again normalized to CALB on the same G500 workloads, with every box aggregating the samples of all three AMS variants. The counters are collected in runs separate from the timing runs and averaged over the same 10 repetitions used elsewhere; since all three policies execute the same logical SpGEMM work with the same accumulator on the same input, their event counts are directly comparable. The three policies are essentially indistinguishable at the L1D level, where every measurement stays within 3% of CALB (Figure 13(a)). This is expected: a private L1D is far too small to hold a morsel's working set under any schedule, so the order in which morsels are dispatched cannot affect it. The divergence emerges deeper in the hierarchy, precisely where concurrently executing morsels begin to share capacity. BGS and LPT incur 1.14–1.28× and 1.07–1.42× more L2 misses than CALB (Figure 13(b)), and LPT additionally exhibits the highest LLC miss counts, with the wider spread of the two policies (Figure 13(c)). The ordering of the two policies in cache misses therefore mirrors their ordering in throughput, with LPT the worst on both counts, which is what the contention explanation predicts and what a purely load-balance-based account cannot produce.

This validates the rationale behind CALB: by dispersing threads across the cf spectrum instead of concentrating them at the high-cf end, CALB keeps the aggregate working set of concurrently processed morsels within the cache capacity, confirming that effective load balancing must account for cache contention. Notably, LPT has been a seminal scheduling strategy for over 50 years [44], and our

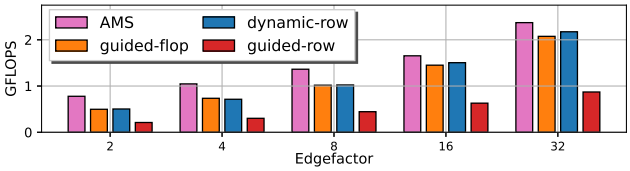


Figure 14: Hashing-based SpGEMM performance on G500 matrices (scale=17) compared to OpenMP dynamic (-row) and guided (-row and -flop) scheduling policies.

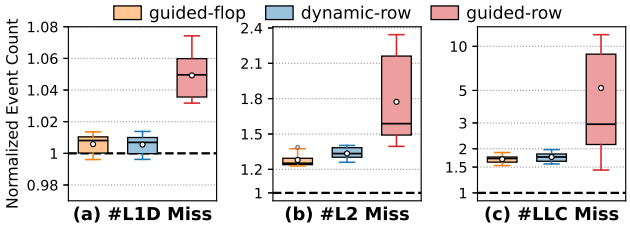


Figure 15: Distribution of the L1D, L2, and LLC misses incurred by the OpenMP dynamic-row, guided-flop, and guided-row scheduling policies of Figure 14, each normalized to the corresponding AMS-based execution (dashed line at 1.0); since these policies schedule a different unit of work, the baseline is AMS as a whole rather than CALB alone. Within each box, the solid line marks the median, the white circle the mean, and the whiskers extend to the most extreme samples within 1.5× the interquartile range. Note the logarithmic y-axis in (b) and (c). (a) L1D misses. (b) L2 misses. (c) LLC misses.

findings highlight the need to revisit and refine classical techniques with the idea of cache pressure alleviation.

We further compare CALB against two widely adopted OpenMP load-balancing strategies: dynamic and guided scheduling [18, 80]. For a fair evaluation, we implemented dynamic scheduling using an empirically determined optimal morsel size of 128 consecutive rows (dynamic-row). Guided scheduling is tested with a minimum morsel set by either 128 rows (guided-row) or a 128-floating-point operation (FLOP) count (guided-flop); both configurations were chosen for the purpose of preserving spatial locality.

As Figure 14 illustrates, CALB achieves higher throughput across all comparisons: 11.87%–48.01% over dynamic-row, 12.89%–47.13% over guided-flop, and a substantial 2.79×–3.59× over guided-row. The guided-row method performs particularly poorly because OpenMP guided scheduling progressively reduces its chunk size, which, similar to LPT [44], degrades spatial locality during later processing stages. This result underscores a critical trade-off: effective parallel scheduling must balance load distribution with the preservation of spatial locality.

The remaining OpenMP methods (dynamic-row and guided-flop) effectively balance skewed workloads and maintain spatial locality, yet still fall short of CALB because they do not account for on-chip cache contention.

Figure 15 profiles the three OpenMP policies with the same hardware performance counters and reproduces the hierarchy-wide pattern of Figure 13. Because these policies schedule a unit of work

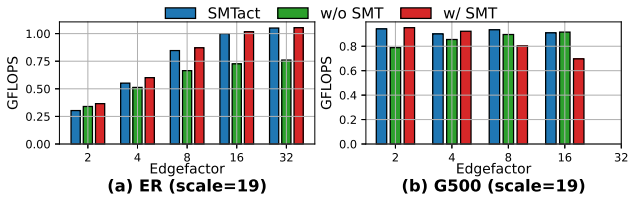


Figure 16: Effectiveness of SMTact in managing SMT to improve AMS-HYB performance on non-skewed (ER) and skewed (G500) workloads.

different from our morsel, they replace AMS's scheduling as a whole rather than CALB alone; the measurements are therefore normalized to AMS rather than to CALB. Guided-flop and dynamic-row stay within roughly 1% of AMS at the L1D level, where the two are too close to order meaningfully, yet their median counts rise to 1.27–1.34× at L2 and to 1.7–1.8× at the shared LLC. Guided-row is both more severe and far more variable: its median is only about 1.05× at L1D but reaches about 1.6× at L2 and 2.9× at LLC, with individual measurements exceeding 10× AMS. The resulting ordering deeper in the hierarchy, with AMS lowest, guided-flop and dynamic-row close behind, and guided-row far above, tracks the throughput ordering of Figure 14. Differences that are negligible in the private L1D are therefore amplified at L2 and most of all at the shared LLC, which is consistent with AMS both preserving spatial locality and limiting shared-cache contention; the miss counts alone do not separate the two mechanisms.

This confirms that the advantage of CALB stems directly from its contention-aware design, which is essential for effective parallel load balancing.

4.3.2 Effectiveness of SMTact. Our SMTact is designed as an adaptive SMT-enablement strategy by balancing the trade-off between increased computational capability and elevated cache pressure. To evaluate its effectiveness, we compare SMTact against two fixed configurations: SMT always enabled (w/ SMT) and SMT always disabled (w/o SMT). The results, presented in Figure 16, include both non-skewed and skewed workloads. As illustrated, neither fixed configuration is uniformly viable: w/ SMT falls up to 23.9% short of the best configuration on the skewed G500 workloads, where the extra logical threads aggravate cache contention, whereas w/o SMT forfeits up to 28.6% on the non-skewed ER workloads that profit from the additional thread-level parallelism. Since SMTact ultimately executes as one of these two configurations, its goal is not to surpass both in every case, but to automatically select a configuration at or close to the best, staying within 8.2% of it in eight of the nine cases. The exception is the extremely sparse ER matrix at *edgefactor* = 2, where SMTact falls 17.1% short of the best fixed configuration because the sorting overhead introduced by CALB is not yet amortized; this penalty is quickly offset as *edgefactor* or matrix scale increases. Given the wide variability in sparsity patterns in real-world SpGEMM workloads, SMTact thus offers a robust default that avoids the substantial degradation either fixed policy incurs outside its sweet spot.

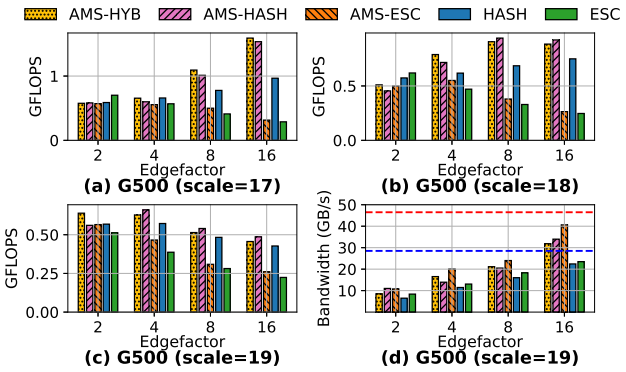


Figure 17: Performance of SpGEMM on G500 workloads. (a)–(c): Measured throughput at G500 scales 17, 18, and 19. (d): Sustained bandwidth measured for different algorithms for the scale-19 workload (the red and blue dashed lines indicate the peak bandwidth of the CXL-augmented memory system (46.50 GB/s) and host DRAM (28.50 GB/s), respectively).

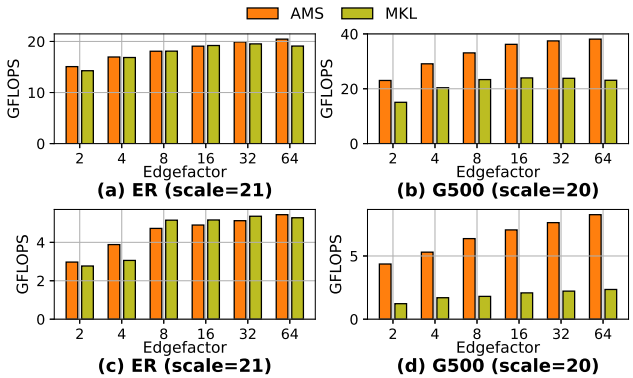


Figure 18: Performance of AMS versus MKL for SpMM (a,b) and SpMV (c,d) operations on ER and G500 workloads.

4.4 Evaluation on Bandwidth-Expanded System

We now evaluate AMS on a bandwidth-expanded memory system, where CXL boosts aggregate bandwidth but introduces higher access latency. As discussed in § 1, load imbalance can inadvertently amplify CXL's latency penalty on straggler threads, negating the bandwidth gain and making effective load balancing particularly critical. For this evaluation, the machine has the same CPU and OS configuration as our previous platform but features host DRAM of 28.50 GB/s bandwidth. Additionally, a CXL expansion memory of 20.95 GB/s bandwidth is installed, yielding a combined system bandwidth of 46.50 GB/s [53, 64, 106].

We evaluate AMS approaches against their vanilla counterparts using skewed (G500) workloads and report the results in Figure 17. As shown, AMS-based methods generally outperform the other methods (Figure 17(a)–(c)). Furthermore, the profiling results confirm that AMS effectively utilizes the additional CXL-augmented bandwidth: AMS-ESC sustains a bandwidth of up to 40.60 GB/s, far exceeding the DRAM-only peak (28.50 GB/s) (Figure 17(d)). In contrast, the bandwidth consumption of the vanilla counterparts does not exceed that of the host DRAM, indicating that AMS indeed improves bandwidth utilization by addressing load imbalance.

We also observe an interesting trend in Figure 17(a)–(c): while AMS-based methods remain competitive on skewed workloads, the performance gap between AMS and the vanilla baselines varies between G500 matrices at scale 17 versus those at scales 18 and 19. Taking the HASH-based variants as an example, AMS-HASH outperforms HASH by 58.77% on the G500 matrix at scale 17, delivering 1.53 GFLOPS versus 0.97 GFLOPS (*edgefactor* 16). However, this advantage diminishes to 23.16% at scale 18, with a similar trend observed at scale 19.

This reduction in the performance gap is mainly attributed to the coincidental load balance occasionally achieved by the static flop-count-based scheme. It is the extent of this imbalance that largely determines the performance degradation of baselines relative to their AMS counterparts. As Figure 4 illustrates, computation and materialization times can exhibit opposing trends across threads. This is precisely the multi-dimensional skew identified in § 2.2: computation and materialization costs trend in opposite directions, and any single-metric scheme can only coincidentally balance them. In some cases, these opposing patterns may randomly offset each other, resulting in a seemingly balanced overall execution time despite the underlying skew. This balance, however, is merely incidental and heavily dependent on the platform's relative sensitivity to computation versus memory access. Consequently, it fails to provide consistent or portable load balance across different architectures.

Previous results on a 179.50 GB/s bandwidth memory corroborate this explanation (cf. Figure 10(g)–(i)). With this higher bandwidth, AMS-HASH achieves a 45.44% performance increase over HASH (1.52 GFLOPS vs. 1.05 GFLOPS) on the G500 matrix at scale 18, because the increased bandwidth significantly reduces the platform's vulnerability to materialization cost. Collectively, these results validate our claim that AMS provides a robust and portable solution to load balancing across diverse systems.

4.5 AMS Evaluation on Other Sparse Matrix Multiplication Kernels

Having established AMS's portability across memory systems, we now turn to its generalizability across kernels. Beyond SpGEMM, we evaluate AMS on other sparse matrix multiplication kernels, specifically, sparse matrix-matrix multiplication (SpMM) and sparse matrix-vector multiplication (SpMV). Since these kernels produce fixed-size dense output, we form morsels by grouping consecutive rows based on a targeted *nnz* count in the left-hand side matrix sub-block (see § 3.5), with each morsel sized at half the L2 cache capacity. The resulting morsels have disparate densities, enabling CALB and SMTact to alleviate cache contention as in SpGEMM.

We compare this AMS-based SpMM and SpMV implementation against the state-of-the-art Intel MKL library on both non-skewed ER and skewed G500 workloads. As shown in Figure 18, our method achieves on-par performance with MKL on non-skewed ER workloads while significantly outperforming it on skewed G500 workloads, with throughput gains of up to 57.25% and 3.51× for SpMM and SpMV, respectively. This demonstrates the broad applicability

and robust advantages of AMS's core scheduling principles across various sparse matrix multiplication kernels.

4.6 Join-Aggregate and Join-Project Query Evaluation

The evaluations so far remain within sparse linear algebra. The remainder of this section returns to the workloads that motivated AMS in § 1: relational queries here, and graph analytics in § 4.7. We first validate AMS on relational workloads to demonstrate its empirical usefulness. Recent studies have shown that relational tables can be modeled as sparse matrices, so that SpGEMM evaluates join-aggregate and join-project queries without materializing the intermediate join result as the traditional join-then-aggregate paradigm does [2, 4, 58], a key advantage for many-to-many joins and self-joins, which produce large intermediate cardinalities. We evaluate both query classes on the same datasets, first join-aggregate and then join-project.

Regarding workloads, we employ widely-used datasets from prior works [32, 54, 91], and also evaluate on an industrial OLAP ClickHouse benchmark [87] with two sampled workloads: CLH0, a uniform 5% sample of users, and CLH1, the top 5% of users by pageviews (see Table 3 for key statistics). These datasets span a wide range of skewness, from low to highly skewed, as quantified by the Gini coefficient in Table 3. Notably, the uniform sample exhibits a higher Gini coefficient than the top-user sample (0.59 versus 0.39), as restricting the population to the heaviest users truncates the long tail, whereas a representative sample preserves the skew inherent in real-world workloads. As for the query form, we perform self-join (i.e., $R = S$) with aggregation in the form:

```
SELECT R.user, S.user, SUM(R.val * S.val)
FROM R, S
WHERE R.item = S.item AND R.user <> S.user
GROUP BY R.user, S.user
```

The corresponding join-project query replaces the aggregation with a projection over the same self-join:

```
SELECT DISTINCT R.user, S.user
FROM R, S
WHERE R.item = S.item AND R.user <> S.user
```

Both queries measure user-wise similarity, where "item" and "val" abstract the shared attribute and measure across scenarios, e.g., URL/click-count in ClickHouse (CLH0 and CLH1) and movie/rating in MovieLens (MVLN). The aggregate form is particularly challenging, as the aggregated values increase the demand for memory, and it is not amenable to existing aggregate query rewriting techniques [24, 35–37, 73, 103, 104, 109, 110].

We compare AMS-HYB with: (1) DIM3 [54], the state-of-the-art join-aggregate method based on SpGEMM; (2) NPHJ-AGG, where a fast hash table [88] performs aggregation on-the-fly alongside the probing phase of non-partitioned hash join (NPHJ) [12, 86], also bypassing intermediate materialization; (3) PHJ-AGG, where we use the cache-efficient partitioned hash join [8–10] to perform the join and materialize the intermediate result, and after that we use the same efficient hash table [88] as NPHJ-AGG to perform the aggregation [19] ; (4) the industrial SpGEMM method: MKL with non-sorted output. For reference, we also include NPHJ and PHJ as join-only baselines (which do not materialize the intermediate results). All evaluated methods are summarized in Table 4 for a clear reference. The join-project setting uses the same set of methods, with the two hash-join variants performing an on-the-fly projection instead of an aggregation, denoted NPHJ-PRJ and PHJ-PRJ.

The two query classes differ in one respect that matters for the SpGEMM-based methods. Following prior work [4, 54], join-project is a Boolean matrix product: only the existence of each output nonzero matters, not its numerical value. We therefore let AMS and DIM3 terminate the accumulation of a cell once it is known to be nonzero, an optimization the join-aggregate form cannot use because every contribution must be summed. The proprietary MKL

[19]We do not have PHJ-AGG perform the on-the-fly aggregation as NPHJ-AGG, because on-the-fly aggregation will incur substantial memory traffic to the aggregation hash table, which will break the cache-level efficiency of partitioning, making this benefit completely disappear.

Table 3: Summary of the 10 relational workloads used for both join-aggregate and join-project evaluation.

Dataset	$\|user\|$	$\|tuple\|$	Gini	$\|OUT_J\|$	$\|OUT_A\|$	$\|OUT_J\|/\|OUT_A\|$
IMDB [55]	733.47K	7.52M	0.3453	214.77M	176.91M	1.2140
ISF18 [77]	1.82K	1.03M	0.5586	327.58M	3.29M	99.6586
CLH0 [87]	850.00K	2.90M	0.5858	2.43B	2.34B	1.0415
RD23 [89]	6.72K	3.91M	0.5276	4.61B	45.02M	102.3613
BKGN [62]	350.33K	5.15M	0.7242	11.43B	6.77B	1.6892
GDBK [112]	53.42K	5.98M	0.1293	19.63B	2.51B	7.8261
CLH1 [87]	850.00K	24.05M	0.3862	20.34B	11.41B	1.7823
JOKE [54]	23.50K	1.71M	0.1709	32.65B	552.25M	59.1219
SAC18 [61]	104.66K	10.00M	0.6988	68.52B	5.94B	11.5325
MVLN [54]	138.49K	20.00M	0.5807	269.61B	16.80B	16.0477

1. $|user|$ and $|tuple|$ are equivalent to the matrix row count and total nnz count, respectively, reflecting the relational-to-sparse-matrix duality; Gini captures the user-wise nnz distribution [63].
2. $|OUT_J|$ and $|OUT_A|$ represent the cardinalities of the join result and the final aggregated result, respectively.

Table 4: Summary of all evaluated join-aggregate query processing approaches

Method	In-place Agg.*	Description
Traditional join followed by aggregation		
`PHJ-AGG [86, 88]`	✗	Radix partitioned hash join with aggregation; deduplication via fast hash table [54, 88].
`NPHJ-AGG [15, 88]`	✓	Non-partitioned hash join; aggregates computed during probing to skip the final deduplication pass [54, 88].
Join-alone algorithms (no aggregation; for comparison only)		
`PHJ [86]`	—	Radix partitioned hash join; serves as the join backend of PHJ-AGG. Does not produce aggregation results.
`NPHJ [15]`	—	Non-partitioned hash join; serves as the join backend of NPHJ-AGG. Does not produce aggregation results.
State-of-the-art join-aggregate techniques		
`DIM3 [54]`	✓	Hybrid sparse/dense join-project query algorithm; intersection-free partitioning eliminates deduplication.
Industry SpGEMM competitors		
`MKL [56]`	✓	Intel oneMKL SpGEMM (non-sorted variant); aggregation replaces deduplication inside the accumulator.
Proposed AMS-based approaches		
`AMS-HYB`	✓	AMS scheduling with hybrid HASH/ESC selection; eliminates the separate deduplication pass.

* *In-place Agg.* denotes whether aggregation is performed on-the-fly alongside the join, or both operations are directly merged into one.

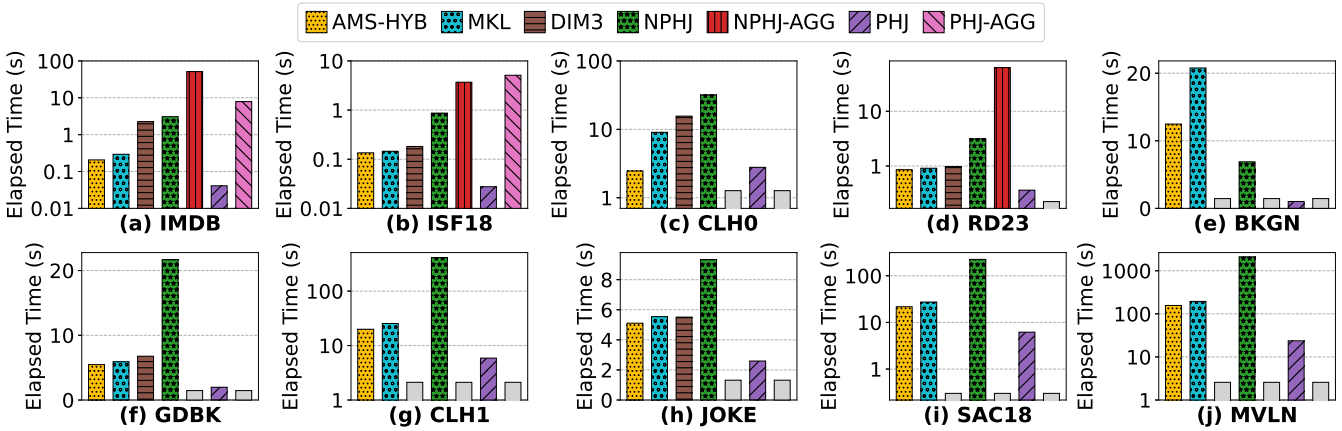


Figure 19: Execution time comparison across diverse join-aggregate workloads; gray bars with "N/A" denote algorithms that failed to complete the execution on those specific workloads; note that vertical axes use a logarithmic scale to capture the wide range of execution times, except for subfigures (e), (f), and (h).

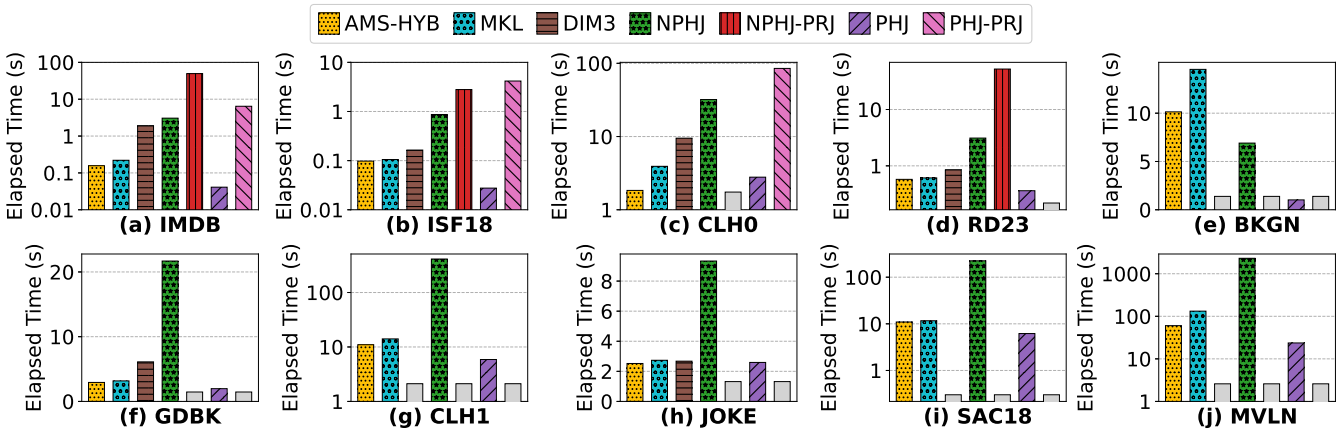


Figure 20: Execution time comparison across diverse join-project workloads; gray bars with "N/A" denote algorithms that failed to complete the execution on those specific workloads; note that vertical axes use a logarithmic scale to capture the wide range of execution times, except for subfigures (e), (f), and (h).

offers only a numerical product and cannot exploit this property in either setting.

Figure 19 presents the join-aggregate results. As shown, AMS-HYB consistently achieves the best performance among all aggregate variants, demonstrating its effectiveness in addressing relational join queries. Surprisingly, even compared with the join-only baseline NPHJ, AMS-HYB wins 9 out of 10 cases, losing only on BKGN, demonstrating the superiority of SpGEMM methods in handling joins with large output cardinalities. Comparing the two runners-up, MKL and DIM3, DIM3 performs slightly worse and cannot finish on 4 workloads (BKGN, CLH1, SAC18, and MVLN) due to out-of-memory errors. This is because DIM3 uses an "estimate-partition-multiplication" approach that tends to over-estimate output size with real-world workloads, which are mostly skewed, and thus DIM3 incurs significant memory materialization cost. Memory is not its only limitation. DIM3 is proposed primarily as an algorithmic remapping, i.e., expressing the query as a sparse matrix product, and leaves the parallelization to a plain OpenMP loop without a scheduling design of its own. It therefore inherits exactly the load imbalance that AMS targets, which is why it generally trails MKL, an industrial standard with mature thread scalability, even where DIM3 completes. Notably, AMS-HYB leads MKL by 1.37× in geometric mean and by up to 3.69× on CLH0, confirming that AMS's ability to address multi-dimensional skew and alleviate on-chip cache contention translates directly to relational workloads where skew is prevalent. The advantage is largest precisely where the aggregation retains most of the join output, as on CLH0 and BKGN ($|OUT_J|/|OUT_A|$ of 1.04 and 1.69), since these are the workloads whose morsels carry the heaviest materialization and thus the most contention to alleviate. Regarding NPHJ-AGG and PHJ-AGG, they finish only on the workloads with the smallest

intermediate cardinalities, three and two of the ten, respectively. This is because, regardless of whether the aggregation is performed on-the-fly or after the join result is fully populated, the hash table must grow dynamically to hold the unhashed entries that are to be aggregated. This dynamically growing hash table is usually resized with a specific scaling factor (typically 2×), and therefore exhausts the memory footprint very quickly. Beyond memory limitations, comparing the "join-then-aggregate" approaches with their respective join-only baselines, we can see that the aggregation-based methods are an order of magnitude slower, indicating the significant cost of aggregation, and thus underscoring the necessity and importance of SpGEMM-based methods. This is particularly true for PHJ and PHJ-AGG: PHJ alone achieves the best runtime on all workloads but CLH0, yet aggregation inflates its runtime by more than two orders of magnitude wherever PHJ-AGG completes (for instance from 0.041 to 7.852 seconds on IMDB). This stark contrast indicates that the aggregation operation itself is the dominant bottleneck, underscoring the need for effective join-aggregate query processing.

Replacing the aggregation with a projection leaves the join, the datasets, and the skew unchanged, and alters only what each method must do with a matched pair once it is found. Figure 20 reports this second setting, and the ranking of the methods carries over unchanged. AMS-HYB again achieves the best performance among all join-project variants and again wins 9 of the 10 cases against the join-only NPHJ, losing only on BKGN. DIM3 remains the closest SpGEMM competitor yet fails on the same four workloads with the largest outputs, for the same two reasons given above. NPHJ-PRJ and PHJ-PRJ complete only on the workloads with the smallest projected output, since the on-the-fly projection hash table must grow dynamically to hold unhashed entries and typically doubles in size until memory is exhausted, and NPHJ-PRJ remains an order of magnitude slower than the join-only NPHJ, confirming that the projection, and not the join, is the dominant cost. Against MKL, AMS-HYB leads by 1.33× in geometric mean and by up to 2.19× on MVLN.

Comparing the two settings isolates what early termination actually buys. Join-project is uniformly the cheaper query for every method that produces output, with AMS-HYB running 1.66× faster under projection than under aggregation in geometric mean, MKL 1.71×, and DIM3 1.34×. The join-only baselines are the control: NPHJ and PHJ perform identical work in both settings and indeed move by less than 1% (0.99× for both), which localizes the entire saving to the post-join stage rather than to the join. AMS's margin over MKL, however, is comparable across the two settings (1.37× against 1.33× in geometric mean). Early termination is therefore not a differentiator on its own: abandoning a cell once it is known to be nonzero removes accumulation work that AMS and DIM3 both exploit, while MKL benefits nearly as much from the smaller Boolean output payload it must materialize. What the projection form does change is the balance between the two methods behind AMS, since the saving falls on high-cf morsels, in which many intermediate products collapse into few output entries, and these are precisely the morsels CALB co-schedules against low-cf ones. The clearest beneficiary is MVLN, where AMS's lead over MKL widens from 1.25× to 2.19×, the largest such shift among the ten workloads.

Table 5: M_l construction on DBLP V10 (seconds).

Meta-path	$\|row\|$	$\|nnz\|$	Gini	AMS-HYB	MKL	Speedup
APCPA	1.77M	4.98M	0.62	0.052	0.117	2.24×
APTPA	1.77M	48.48M	0.60	0.158	0.529	3.34×
TPAPT	120.69K	48.48M	0.94	0.235	0.382	1.62×
CPAPC	5.08K	4.98M	0.78	0.067	0.120	1.80×
Geomean	–	–	–	0.107	0.231	2.16×

DIM3 is the one method that gains least from the cheaper form (1.34× against MKL's 1.71×), and its gap to MKL correspondingly widens under projection, for instance from 1.72× to 2.42× on CLH0 and from 1.14× to 1.93× on GDBK. This is consistent with the diagnosis above: with less work to perform per matched pair, scheduling inefficiency accounts for a larger share of DIM3's runtime, so relying on a plain OpenMP loop costs it more, not less, as the kernel becomes cheaper.

4.7 Graph-Analytic Evaluation

Graph analytics is among the primary consumers of SpGEMM, and its workloads are typically built on real-world graphs whose degree distributions are heavily skewed. We therefore evaluate AMS on four graph-analytic workloads: meta-path construction for PathSim similarity search, breadth-first search, triangle counting, and an end-to-end Markov clustering pipeline.

4.7.1 PathSim Meta-Path Construction. We first evaluate AMS on the construction of intermediate meta-path matrices used by PathSim [93], a meta-path-based similarity measure for heterogeneous information networks. Given a symmetric meta-path, PathSim scores node pairs through a commuting matrix $M = M_l M_l^T$, whose left half M_l is a chain of sparse products. Constructing M_l is therefore an SpGEMM workload, and it dominates the cost of answering a query.

We evaluate on the AMiner DBLP V10 citation network [96], comprising 1.77M authors (A), 3.08M papers (P), 5,078 venues (C), and 120,685 terms (T), with 9.48M author-paper, 2.57M venue-paper, and 22.64M paper-term edges. We select four symmetric meta-paths that place each node type in the row dimension: APCPA and APTPA relate authors through shared venues and terms, whereas TPAPT and CPAPC relate terms and venues through shared authors. As Table 5 summarizes, they differ widely in both shape and skew, and form two transpose pairs (APCPA/CPAPC and APTPA/TPAPT) that perform the same work over row counts differing by more than two orders of magnitude. All four exhibit pronounced row-wise skew (Gini 0.60–0.94), reflecting prevalent skew in real-world graphs. We report the SpGEMM time of the M_l construction.

AMS-HYB outperforms MKL on all four meta-paths, by 1.62× to 3.34×, giving a geometric-mean speedup of 2.16×. These gains span a 350× range of row counts, from the 5,078-row CPAPC to the 1.77M-row APTPA, indicating that the benefit of AMS does not depend on a particular matrix shape. Within each transpose pair, however, the gain is consistently larger for the variant with more rows (2.24× against 1.80×, and 3.34× against 1.62×), as a larger row count yields finer morsel granularity for CALB to redistribute and better amortizes the cost of cf-based morsel sorting. Overall, AMS

extends beyond matrix squaring and relational queries to meta-path construction on heterogeneous networks.

4.7.2 Breadth-First Search. We next evaluate AMS on multi-source breadth-first search [33], a building block of betweenness centrality and of distributed Markov clustering. Expressed in the linear-algebraic formulation, a BFS step multiplies a frontier matrix, whose columns hold the sources being explored, by the adjacency matrix, so that each traversal level is one SpGEMM. As Figure 21 shows, we sweep the graph *scale* over 17–19, the number of concurrent sources over 2^{10}–2^{16}, and the *edgefactor* over 2–32, and compare AMS-HYB and AMS-HASH against the HASH baseline and Intel MKL. We follow the setup of Nagasaka et al. [76], who cast this kernel as the product of a square graph matrix with a tall-skinny frontier matrix and evaluate it on generated R-MAT graphs rather than on a fixed collection. Synthetic graphs are the appropriate choice here because the frontier width is itself an experimental variable: the number of concurrent sources sets the number of columns of the right-hand operand, and sweeping it independently of graph size and density is only possible with a generator. We extend that setup by sweeping *edgefactor* as well, so that density varies alongside frontier width. Frontier expansion also imposes no ordering requirement on its output, as a frontier is consumed as a set of reached vertices, so we report the non-sorted variants here, which is what a real deployment would run. Running a sorted variant would only add the extra instructions that producing row-wise ordered output requires (§ 4.2), with no benefit to the application.

Across the 60 configurations of Figure 21, AMS-HYB leads the HASH baseline by 1.18× in geometric mean and by up to 1.63×, ahead in 50 of them, while AMS-HASH leads by 1.13×; against MKL the margin is far wider, 1.91× in geometric mean and up to 3.31×. Two trends are visible across the panels. The advantage grows monotonically with density, from 0.96× at *edgefactor* 2 to 1.07×, 1.21×, 1.33×, and 1.38× at *edgefactor* 32, and the single regime in which AMS does not pay off is the sparsest one, where the cf-based morsel sorting is not yet amortized, exactly as observed for matrix squaring in § 4.2. The advantage also grows with the number of concurrent sources, from about 1.05× at 2^{10} sources to about 1.28× at 2^{14}, because a wider frontier makes each traversal level denser and thus concentrates more contention for CALB to relieve. Both trends confirm that the gains follow the amount of skew-induced contention actually present, rather than being an artifact of any particular graph size.

4.7.3 Triangle Counting. We then evaluate AMS on triangle counting [95], which counts the triangles of an undirected graph through the masked product $L \cdot U$, where L and U are the strictly lower and upper triangular parts of the adjacency matrix. Since the masking step consumes row-wise ordered output, we report the sorted variants only, comparing AMS-ESC against the ESC baseline and the sorted MKL variant (MKLS) on nine square SuiteSparse matrices, following the evaluation setup of Nagasaka et al. [76], as reported in Figure 22. Real matrices, rather than generated ones, are the appropriate choice for this kernel: the input is the adjacency matrix of an actual undirected graph, and the pipeline reorders its rows by increasing nonzero count before splitting it into L and U, so the workload is defined by a real graph's structure and by what that preprocessing does to it. Nagasaka et al. [76] observe that this preprocessing destroys the block structure of the original matrix and thereby lowers the compression ratio relative to squaring, which in turn makes the sorting of the output account for a larger share of execution time. Triangle counting is therefore not merely a workload that tolerates sorted output but one in which sorting is a substantial cost, which is what makes it the informative case for the sorted regime. Together with the non-sorted BFS results above, the two workloads cover both output-ordering regimes, each reported in the form its application actually requires.

Table 6: Summary of HipMCL Datasets: Viruses and Archaea [7, 76]

Dataset	$\|vertices\|$	$\|edges\|$	Gini	Iters. to Converge*
Viruses	219,715	9,166,070	0.6574	37
Archaea	1,644,227	409,568,764	0.5759	51

*Number of iterations until the algorithm converges to find clusters.

As Figure 22 shows, AMS-ESC outperforms the ESC baseline on eight of the nine matrices, by 1.27× in geometric mean and by up to 1.49×, and outperforms MKLS on all nine, by 1.55× in geometric mean and by up to 2.77×. This result is notable because the sorted regime is the less favorable one for AMS: as § 4.2 shows, ESC's radix sort has a higher and more predictable per-row cost than hashing, which both narrows the headroom that better scheduling can recover and makes flop-count-based partitioning a closer approximation of the true cost. That AMS still recovers a consistent margin here indicates that the contention it removes is not an artifact of hash-table probing, but a property of how skewed morsels share the cache under any accumulator.

4.7.4 Markov Clustering. We finally evaluate AMS within an end-to-end scientific computing pipeline, namely the HipMCL [7, 76] Markov clustering benchmark. HipMCL identifies clusters in protein-similarity networks by simulating concurrent random walks over the graph, which are realized as repeated sparse matrix squaring of the similarity matrix until a stable clustering structure emerges. Across iterations, the similarity matrix becomes progressively sparser as the clustering structure crystallizes, yet sparse matrix squaring still dominates the overall runtime. We therefore integrate our SpGEMM implementations into HipMCL and report both the SpGEMM kernel time and the total end-to-end execution time on the two HipMCL datasets summarized in Table 6.

Figure 23 reports the results. AMS-HYB and AMS-HASH consistently deliver the best performance on both workloads, owing to their effective load balancing and on-chip cache contention alleviation. Compared with the HASH baseline, AMS-HASH reduces SpGEMM kernel runtime by 47.15% on Viruses and 20.91% on Archaea, translating into 10.63% and 9.78% improvements in end-to-end HipMCL execution time, respectively. The larger gain on Viruses can be attributed to two factors: (1) Viruses exhibits a slightly more skewed similarity structure (Gini = 0.6574 vs. 0.5759), which amplifies the benefits of AMS's skew-aware scheduling; and (2) Archaea requires substantially more MCL iterations to converge, and the later iterations operate on increasingly sparse matrices where load imbalance and cache contention become less pronounced, thus diluting AMS's relative advantage. For ESC-based

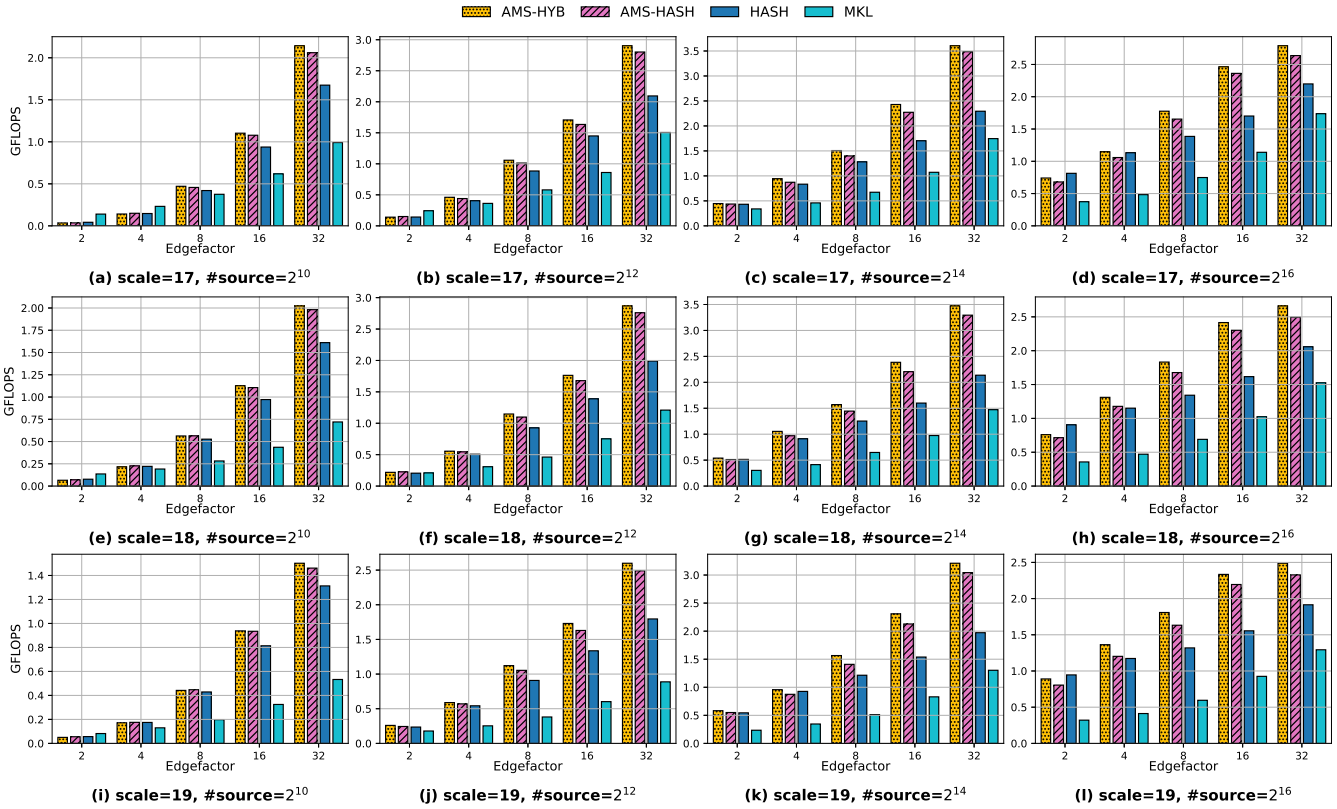


Figure 21: Multi-source BFS throughput on R-MAT graphs, where a frontier matrix holding *#source* sources is repeatedly multiplied by the adjacency matrix. Subfigures (a)–(l) sweep the graph *scale* over 17–19 and the number of sources over 2^{10}–2^{16}, with *edgefactor* varying along the x-axis of every panel.

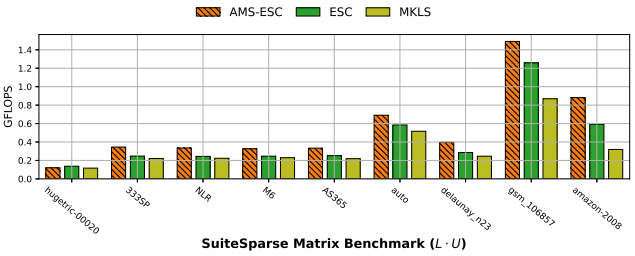


Figure 22: Triangle counting throughput on nine SuiteSparse matrices, obtained from the masked product $L \cdot U$ of the strictly lower and upper triangular parts of each adjacency matrix. Only sorted variants are reported, as the masked formulation requires row-wise ordered output.

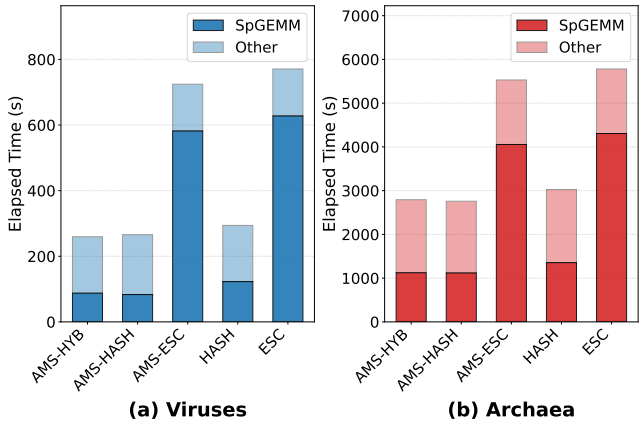


Figure 23: Comparison of HipMCL [7] execution times for (a) Viruses and (b) Archaea datasets. Each bar is partitioned into the time spent on the specific SpGEMM kernel (dark) and the overhead from the remaining HipMCL framework (light).

methods, the requirement of producing sorted output incurs noticeably higher SpGEMM execution time overall; nevertheless, AMS-ESC still outperforms the ESC baseline thanks to its superior load balancing and cache contention mitigation. Finally, AMS-HYB closely matches AMS-HASH on both workloads, confirming that HYB reliably selects the appropriate accumulator on a per-morsel basis even under the rapidly evolving sparsity patterns of MCL iterations.

4.8 Discussion

Our experimental analysis across SpGEMM, other sparse matrix kernels, relational join-aggregate and join-project queries, and graph-analytic workloads, including meta-path construction, multi-source breadth-first search, triangle counting, and end-to-end Markov clustering, on diverse workloads and hardware platforms reveals several key findings that collectively validate the design rationale and practical effectiveness of AMS.

Multi-dimensional skew is the primary performance bottleneck. Across all SpGEMM evaluations (§ 4.2), we observe that the performance gap between AMS and flop-count-based baselines widens as workload skew increases. On non-skewed ER matrices, AMS performs comparably to the baselines, confirming that it introduces negligible overhead. On skewed G500 matrices, however, AMS delivers substantial gains (up to 81.01% for HASH), demonstrating that multi-dimensional skew, not algorithmic inefficiency, is the dominant factor limiting existing methods. That said, AMS may occasionally underperform in highly sparse cases due to overhead from cf-based morsel sorting; however, this penalty diminishes rapidly as matrix density increases, and is outweighed by the substantial gains on the skewed workloads that dominate real-world applications.

Contention-aware scheduling is essential, not optional. The dissection of CALB and SMTact (§ 4.3) shows that simply achieving load balance is insufficient: methods that balance load but ignore cache contention (e.g., OpenMP dynamic and guided scheduling) still fall short of CALB. Similarly, SMTact's selective SMT activation outperforms both always-on and always-off SMT configurations, confirming that the trade-off between computational throughput and cache pressure must be managed explicitly rather than left to a fixed policy.

Load balancing unlocks hardware potential. The CXL evaluation (§ 4.4) demonstrates that load imbalance can negate the benefits of bandwidth expansion, as straggler threads are disproportionately penalized by CXL's higher access latency. By addressing this imbalance, AMS enables effective utilization of the augmented bandwidth, achieving throughput that exceeds the DRAM-only peak. This finding has broader implications: as heterogeneous memory systems become more prevalent, contention-aware load balancing will be increasingly critical for realizing their performance potential.

SpGEMM-based methods are superior for relational query processing. The join-aggregate and join-project evaluation (§ 4.6) reveals that traditional hash-join-based approaches, whether producing the output on-the-fly or after materialization, suffer from severe memory scalability issues, completing on only the few workloads with the smallest output. In contrast, SpGEMM-based methods bypass intermediate materialization entirely, and AMS-HYB further improves upon the state of the art by addressing skew in relational workloads, leading MKL by 1.37× and 1.33× in geometric mean on the two query classes and beating even the join-only baseline on 9 of the 10 workloads. Contrasting the two classes is instructive in its own right: the join-only baselines perform identical work under both and indeed move by less than 1%, which localizes the entire difference to the post-join stage, whereas every method that produces output runs faster under projection. The exception is DIM3, which gains the least and whose gap to MKL widens as the kernel becomes cheaper, illustrating that an algorithmic mapping alone is insufficient without a scheduling design to match.

The gains hold in both output-ordering regimes. Graph analytics exercises AMS under both output conventions, and we evaluate each in the form its application requires: non-sorted for multi-source BFS (§ 4.7.2), whose frontier is consumed as a set, and sorted for triangle counting (§ 4.7.3), whose masked product needs row-wise ordered output. AMS leads in both, by 1.18× in geometric mean over the HASH baseline on BFS and by 1.27× over ESC on triangle counting. The sorted regime is the less favorable of the two, since sorting is a higher and more predictable per-row cost that both narrows the recoverable headroom and makes flop-count partitioning a closer approximation of the truth, so a consistent margin there indicates that the contention AMS removes is a property of how skewed morsels share the cache rather than an artifact of any one accumulator. The BFS sweep also shows the margin growing monotonically with density and with the number of concurrent sources, tracking the amount of contention actually present.

End-to-end gains carry over to graph analytic pipelines. The HipMCL evaluation (§ 4.7.4) shows that AMS's SpGEMM-level improvements translate into measurable end-to-end speedups in a real-world Markov clustering pipeline, even though the underlying matrices become progressively sparser across iterations. The larger gains observed on the more skewed Viruses dataset are consistent with our earlier findings, reinforcing that the practical value of contention-aware scheduling grows with workload skew.

The framework generalizes beyond SpGEMM. The evaluation on SpMM and SpMV (§ 4.5) confirms that AMS's core scheduling principles (morsel-wise dynamic scheduling, contention-aware load balancing, and selective SMT activation) are not specific to SpGEMM but apply broadly to sparse matrix multiplication kernels that exhibit irregular sparsity patterns.

5 RELATED WORK

SpGEMM has been extensively studied as a critical kernel in scientific computing and graph analytics [30, 78], and has more recently emerged in relational query processing [32, 49, 50, 54, 91]. These relational works, however, are largely confined to the algorithmic mapping from queries to sparse matrix products, and address neither the efficient parallel execution of the resulting kernel nor the load imbalance under skew that AMS targets. Over the decades, Gustavson's row-row formulation [47] has become the dominant approach due to its natural parallelizability, particularly when paired with the dense array accumulator known as SPA [42]. However, SPA has been shown to be inefficient and to incur high memory overhead, which prompted the development of more memory-efficient alternatives such as hash tables [76, 83], heaps [75], and radix sort [30].

In addition to algorithmic advances, much attention has been given to the challenges of load balancing in parallel SpGEMM. Schemes based on flop counts [75, 76] have emerged as a dominant solution, along with other static strategies such as methods using blocks [19, 20] or graph partitioning [3]. While effective in certain

scenarios, these approaches often fail to account for the multi-dimensional skew characteristics present in real-world SpGEMM workloads, resulting in imbalanced execution.

As a more adaptive alternative, dynamic scheduling offers greater flexibility. For instance, Patwary et al. [79] proposed a dynamic task scheduling scheme that partitions work across both row and column dimensions. However, task partitioning across two dimensions introduces substantial synchronization overhead, which limits its practical applicability. Similarly, techniques leveraging work-stealing [16, 17, 111] and cache-access modeling [46] have been explored, but either incur significant cache coherence overhead or fail to account for complicated LLC sharing characteristics, making them less favorable than approaches based on flop counts. In another line of work, researchers have adapted dynamic variants of the classical longest-processing-time-first (LPT) algorithm [44], which prioritizes heavy tasks over lighter ones. Critically, none of these approaches account for on-chip cache contention caused by concurrently executing dense submatrices, which AMS explicitly addresses alongside multi-dimensional skew.

6 CONCLUSION

This paper presents adaptive morsel scheduling (AMS), a morsel-wise dynamic scheduling framework that addresses two co-existing challenges in SpGEMM: *multi-dimensional skew* and *on-chip cache contention*. AMS balances skewed workloads without aggravating cache contention through contention-aware load balancing (CALB), judiciously activates SMT through data-conscious simultaneous multithreading activator (SMTact), and adaptively selects the per-morsel accumulator through hybrid accumulation mechanism (HYB). Evaluations across matrix squaring, relational join-aggregate and join-project queries, graph analytics, and other sparse kernels show that AMS significantly outperforms state-of-the-art methods on skewed workloads, maintains comparable performance on non-skewed ones, and improves bandwidth utilization on a CXL-expanded memory system, with these gains generalizing across hardware platforms. Looking ahead, incorporating other SpGEMM formulations (e.g., outer-product or inner-product) into the morsel-wise scheduling framework presents a promising direction for further improvements, particularly on highly sparse matrices.

REFERENCES

[1] 2018. VLSI/vas_stokes_1M. https://sparse.tamu.edu/VLSI/vas_stokes_1M

[2] Christopher R. Aberger, Andrew Lamb, Kunle Olukotun, and Christopher Ré. 2018. LevelHeaded: A Unified Engine for Business Intelligence and Linear Algebra Querying. In *ICDE*. IEEE Computer Society, 449–460.

[3] Kadir Akbudak and Cevdet Aykanat. 2014. Simultaneous Input and Output Matrix Partitioning for Outer-Product-Parallel Sparse Matrix-Matrix Multiplication. *SIAM J. Sci. Comput.* 36, 5 (2014).

[4] Rasmus Resen Amossen and Rasmus Pagh. 2009. Faster join-projects and sparse matrix multiplications. In *Proceedings of the 12th International Conference on Database Theory*. 121–126.

[5] Junya Arai, Hiroaki Shiokawa, Takeshi Yamamuro, Makoto Onizuka, and Sotetsu Iwamura. 2016. Rabbit Order: Just-in-Time Parallel Reordering for Fast Graph Analysis. In *IPDPS*. IEEE Computer Society, 22–31.

[6] Ariful Azad, Grey Ballard, Aydin Buluç, James Demmel, Laura Grigori, Oded Schwartz, Sivan Toledo, and Samuel Williams. 2016. Exploiting Multiple Levels of Parallelism in Sparse Matrix-Matrix Multiplication. *SIAM J. Sci. Comput.* 38, 6 (2016).

[7] Ariful Azad, Georgios A Pavlopoulos, Christos A Ouzounis, Nikos C Kyrpides, and Aydin Buluç. 2018. HipMCL: a high-performance parallel implementation of the Markov clustering algorithm for large-scale networks. *Nucleic acids research* 46, 6 (2018), e33–e33.

[8] Cagri Balkesen, Gustavo Alonso, Jens Teubner, and M. Tamer Özsu. 2013. Multi-Core, Main-Memory Joins: Sort vs. Hash Revisited. *Proc. VLDB Endow.* 7, 1 (2013), 85–96.

[9] Cagri Balkesen, Jens Teubner, Gustavo Alonso, and M. Tamer Özsu. 2013. Main-memory hash joins on multi-core CPUs: Tuning to the underlying hardware. In *ICDE*. IEEE Computer Society, 362–373.

[10] Cagri Balkesen, Jens Teubner, Gustavo Alonso, and M. Tamer Özsu. 2015. Main-Memory Hash Joins on Modern Processor Architectures. *IEEE Trans. Knowl. Data Eng.* 27, 7 (2015), 1754–1766.

[11] Grey Ballard, Alex Druinsky, Nicholas Knight, and Oded Schwartz. 2016. Hypergraph Partitioning for Sparse Matrix-Matrix Multiplication. *ACM Trans. Parallel Comput.* 3, 3 (2016), 18:1–18:34.

[12] Maximilian Bandle, Jana Giceva, and Thomas Neumann. 2021. To Partition, or Not to Partition, That is the Join Question in a Real System. In *SIGMOD Conference*. ACM, 168–180.

[13] Richard Barrett, Michael Berry, Tony F Chan, James Demmel, June Donato, Jack Dongarra, Victor Eijkhout, Roldan Pozo, Charles Romine, and Henk Van der Vorst. 1994. *Templates for the solution of linear systems: building blocks for iterative methods*. SIAM.

[14] Nathan Bell, Steven Dalton, and Luke N Olson. 2012. Exposing fine-grained parallelism in algebraic multigrid methods. *SIAM Journal on Scientific Computing* 34, 4 (2012), C123–C152.

[15] Spyros Blanas, Yinan Li, and Jignesh M. Patel. 2011. Design and evaluation of main memory hash join algorithms for multi-core CPUs. In *SIGMOD Conference*. ACM, 37–48.

[16] Robert D. Blumofe. 1994. Scheduling Multithreaded Computations by Work Stealing. In *FOCS*. IEEE Computer Society, 356–368.

[17] Robert D. Blumofe, Christopher F. Joerg, Bradley C. Kuszmaul, Charles E. Leiserson, Keith H. Randall, and Yuli Zhou. 1995. Cilk: An Efficient Multithreaded Runtime System. In *PPoPP*. ACM, 207–216.

[18] OpenMP Architecture Review Board. 1997. OpenMP Fortran Application Program Interface. (1997).

[19] Aydin Buluç and John R. Gilbert. 2008. Challenges and Advances in Parallel Sparse Matrix-Matrix Multiplication. In *ICPP*. IEEE Computer Society, 503–510.

[20] Aydin Buluç and John R. Gilbert. 2008. On the representation and multiplication of hypersparse matrices. In *IPDPS*. IEEE, 1–11.

[21] Aydin Buluç and John R. Gilbert. 2011. The Combinatorial BLAS: design, implementation, and applications. *Int. J. High Perform. Comput. Appl.* 25, 4 (2011), 496–509.

[22] Aydin Buluç and John R. Gilbert. 2012. Parallel Sparse Matrix-Matrix Multiplication and Indexing: Implementation and Experiments. *SIAM J. Sci. Comput.* 34, 4 (2012).

[23] Deepayan Chakrabarti, Yiping Zhan, and Christos Faloutsos. 2004. R-MAT: A Recursive Model for Graph Mining. In *SDM*. SIAM, 442–446.

[24] Surajit Chaudhuri and Kyuseok Shim. 1994. Including Group-By in Query Optimization. In *VLDB'94, Proceedings of 20th International Conference on Very Large Data Bases, September 12-15, 1994, Santiago de Chile, Chile*, Jorge B. Bocca, Matthias Jarke, and Carlo Zaniolo (Eds.). Morgan Kaufmann, 354–366.

[25] YuAng Chen and Yeh-Ching Chung. 2022. Workload Balancing via Graph Reordering on Multicore Systems. *IEEE Trans. Parallel Distributed Syst.* 33, 5 (2022), 1231–1245.

[26] Yu Chen and Ke Yi. 2017. Two-Level Sampling for Join Size Estimation. In *SIGMOD Conference*. ACM, 759–774.

[27] Helin Cheng, Wenxuan Li, Yuechen Lu, and Weifeng Liu. 2023. HASpGEMM: Heterogeneity-Aware Sparse General Matrix-Matrix Multiplication on Modern Asymmetric Multicore Processors. In *ICPP*. ACM, 807–817.

[28] Transaction Processing Performance Council. 2021. TPC BENCHMARKTM DS Standard Specification Revision 3.2.0. https://www.tpc.org/tpc_documents_current_versions/pdf/tpc-ds_v3.2.0.pdf

[29] Leonardo Dagum and Ramesh Menon. 1998. OpenMP: an industry standard API for shared-memory programming. *IEEE computational science and engineering* 5, 1 (1998), 46–55.

[30] Steven Dalton, Luke Olson, and Nathan Bell. 2015. Optimizing sparse matrix—matrix multiplication for the gpu. *ACM Transactions on Mathematical Software (TOMS)* 41, 4 (2015), 1–20.

[31] Timothy A. Davis and Yifan Hu. 2011. The university of Florida sparse matrix collection. *ACM Trans. Math. Softw.* 38, 1 (2011), 1:1–1:25.

[32] Shaleen Deep, Xiao Hu, and Paraschos Koutris. 2020. Fast join project query evaluation using matrix multiplication. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 1213–1223.

[33] Laxman Dhulipala, Charles McGuffey, Hongbo Kang, Yan Gu, Guy E. Blelloch, Phillip B. Gibbons, and Julian Shun. 2020. Sage: Parallel Semi-Asymmetric Graph Algorithms for NVRAMs. *Proc. VLDB Endow.* 13, 9 (2020), 1598–1613.

[34] Kayhan Dursun, Carsten Binnig, Ugur Çetintemel, Garret Swart, and Weiwei Gong. 2019. A Morsel-Driven Query Execution Engine for Heterogeneous Multi-Cores. *Proc. VLDB Endow.* 12, 12 (2019), 2218–2229.

[35] Marius Eich, Pit Fender, and Guido Moerkotte. 2018. Efficient generation of query plans containing group-by, join, and groupjoin. *VLDB J.* 27, 5 (2018),

617–641.
[36] Philipp Fent, Altan Birler, and Thomas Neumann. 2023. Practical planning and execution of groupjoin and nested aggregates. *VLDB J.* 32, 6 (2023), 1165–1190.
[37] Philipp Fent and Thomas Neumann. 2021. A Practical Approach to Groupjoin and Nested Aggregates. *Proc. VLDB Endow.* 14, 11 (2021), 2383–2396.
[38] Jianhua Gao, Weixing Ji, Fangli Chang, Shiyu Han, Bingxin Wei, Zeming Liu, and Yizhuo Wang. 2023. A Systematic Survey of General Sparse Matrix-matrix Multiplication. *ACM Comput. Surv.* 55, 12 (2023), 244:1–244:36.
[39] Jianhua Gao, Bingjie Liu, Weixing Ji, and Hua Huang. 2024. A Systematic Literature Survey of Sparse Matrix-Vector Multiplication. *CoRR* abs/2404.06047 (2024).
[40] Sen Gao, Shengliang Lu, Shixuan Sun, Yuchen Li, and Bingsheng He. 2025. An Efficient Memoization Engine for Concurrent Graph Query Processing. In *ICDE*. IEEE, 1015–1028.
[41] Amir Gholami, Zhewei Yao, Sehoon Kim, Coleman Hooper, Michael W. Mahoney, and Kurt Keutzer. 2024. AI and Memory Wall. *IEEE Micro* 44, 3 (2024), 33–39.
[42] John R Gilbert, Cleve Moler, and Robert Schreiber. 1992. Sparse matrices in MATLAB: Design and implementation. *SIAM journal on matrix analysis and applications* 13, 1 (1992), 333–356.
[43] John R. Gilbert, Steven P. Reinhardt, and Viral B. Shah. 2006. High-Performance Graph Algorithms from Parallel Sparse Matrices. In *PARA (Lecture Notes in Computer Science)*, Vol. 4699. Springer, 260–269.
[44] Ronald L. Graham. 1969. Bounds on Multiprocessing Timing Anomalies. *SIAM Journal of Applied Mathematics* 17, 2 (1969), 416–429.
[45] Zhixiang Gu, Jose Moreira, David Edelsohn, and Ariful Azad. 2020. Bandwidth Optimized Parallel Algorithms for Sparse Matrix-Matrix Multiplication using Propagation Blocking. In *SPAA*. ACM, 293–303.
[46] Jihu Guo, Rui Xia, Jie Liu, Xiaoxiong Zhu, and Xiang Zhang. 2024. CAMLB-SpMV: An Efficient Cache-Aware Memory Load-Balancing SpMV on CPU. In *ICPP*. ACM, 640–649.
[47] Fred G. Gustavson. 1978. Two Fast Algorithms for Sparse Matrices: Multiplication and Permuted Transposition. *ACM Trans. Math. Softw.* 4, 3 (1978), 250–269.
[48] Chuhe Hong, Qinglin Wang, Runzhang Mao, Yuechao Liang, Rui Xia, and Jie Liu. 2024. SaSpGEMM: Sorting-Avoiding Sparse General Matrix-Matrix Multiplication on Multi-Core Processors. In *ICPP*. ACM, 1166–1175.
[49] Xiao Hu. 2025. Output-optimal algorithms for join-aggregate queries. *Proceedings of the ACM on Management of Data* 3, 2 (2025), 1–27.
[50] Xiao Hu and Ke Yi. 2020. Parallel algorithms for sparse matrix multiplication and join-aggregate queries. In *Proceedings of the 39th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*. 411–425.
[51] Yu-Ching Hu, Yuliang Li, and Hung-Wei Tseng. 2022. TCUDB: Accelerating Database with Tensor Processors. In *SIGMOD Conference*. ACM, 1360–1374.
[52] Wentao Huang, Mian Lu, and Kian-Lee Tan. 2026. Hash Joins Meet CXL: A Fresh Look. In *CIDR*. www.cidrdb.org.
[53] Wentao Huang, Mo Sha, Mian Lu, Yuqiang Chen, Bingsheng He, and Kian-Lee Tan. 2024. Bandwidth Expansion via CXL: A Pathway to Accelerating In-Memory Analytical Processing. In *VLDB Workshops*. VLDB.org.
[54] Zichun Huang and Shimin Chen. 2022. Density-optimized intersection-free mapping and matrix multiplication for join-project operations. *Proceedings of the VLDB Endowment* 15, 10 (2022), 2244–2256.
[55] IMDb.com, Inc. 2024. IMDb Non-Commercial Datasets. https://developer.imdb.com/non-commercial-datasets/
[56] Intel. 2024. Intel® oneAPI Math Kernel Library (oneMKL). https://www.intel.com/content/www/us/en/developer/tools/oneapi/onemkl.html
[57] Myung-Hwan Jang, Yunyong Ko, Hyuck-Moo Gwon, Ikhyeon Jo, Yongjun Park, and Sang-Wook Kim. 2023. SAGE: a storage-based approach for scalable and efficient sparse generalized matrix-matrix multiplication. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. 923–933.
[58] Mahmoud Abo Khamis, Xiao Hu, and Dan Suciu. 2025. Fast Matrix Multiplication meets the Submodular Width. *Proc. ACM Manag. Data* 3, 2 (2025), 98:1–98:26.
[59] Farzad Khorasani, Rajiv Gupta, and Laxmi N. Bhuyan. 2015. Scalable SIMD-Efficient Graph Processing on GPUs. In *Proceedings of the 24th International Conference on Parallel Architectures and Compilation Techniques (PACT '15)*. 39–50.
[60] Changkyu Kim, Eric Sedlar, Jatin Chhugani, Tim Kaldewey, Anthony D. Nguyen, Andrea Di Blas, Victor W. Lee, Nadathur Satish, and Pradeep Dubey. 2009. Sort vs. Hash Revisited: Fast Join Implementation on Modern Multi-Core CPUs. *Proc. VLDB Endow.* 2, 2 (2009), 1378–1389.
[61] Denis Kotkov, Joseph A. Konstan, Qian Zhao, and Jari Veijalainen. 2018. Investigating serendipity in recommender systems based on real user feedback. In *SAC*. ACM, 1341–1350.
[62] Denis Kotkov, Alan Medlar, Alexandr Maslov, Umesh Raj Satyal, Mats Neovius, and Dorota Glowacka. 2022. The Tag Genome Dataset for Books. In *Proceedings of the 2022 ACM SIGIR Conference on Human Information Interaction and Retrieval (CHIIR '22)*. 5. https://doi.org/10.1145/3498366.3505833
[63] Jérôme Kunegis and Julia Preusse. 2012. Fairness on the web: alternatives to the power law. In *WebSci*. ACM, 175–184.
[64] Georgiy Lebedev, Hamish Nicholson, Musa Ünal, Sanidhya Kashyap, and Anastasia Ailamaki. 2025. Demystifying CXL Memory Bandwidth Expansion for Analytical Workloads. In *VLDB Workshops*. VLDB.org.
[65] Suhyun Lee, Chaemin Lim, Jinwoo Choi, Heelim Choi, Chan Lee, Yongjun Park, Kwanghyun Park, Hanjun Kim, and Youngsok Kim. 2024. SPID-Join: A Skew-resistant Processing-in-DIMM Join Algorithm Exploiting the Bank- and Rank-level Parallelisms of DIMMs. *Proc. ACM Manag. Data* 2, 6 (2024), 251:1–251:27.
[66] Viktor Leis, Peter A. Boncz, Alfons Kemper, and Thomas Neumann. 2014. Morsel-driven parallelism: a NUMA-aware query evaluation framework for the many-core age. In *SIGMOD Conference*. ACM, 743–754.
[67] Alberto Lerner and Gustavo Alonso. 2024. CXL and the Return of Scale-Up Database Engines. *Proc. VLDB Endow.* 17, 10 (2024), 2568–2575.
[68] Zhonggen Li, Xiangyu Ke, Yifan Zhu, Yunjun Gao, and Yaofeng Tu. 2025. HC-SpMM: Accelerating Sparse Matrix-Matrix Multiplication for Graphs with Hybrid GPU Cores. In *ICDE*. IEEE, 501–514.
[69] Chunxu Lin, Wensheng Luo, Yixiang Fang, Chenhao Ma, Xilin Liu, and Yuchi Ma. 2024. On Efficient Large Sparse Matrix Chain Multiplication. *Proc. ACM Manag. Data* 2, 3 (2024), 156.
[70] Jinshu Liu, Hamid Hadian, Yuyue Wang, Daniel S. Berger, Marie Nguyen, Xun Jian, Sam H. Noh, and Huaicheng Li. 2025. Systematic CXL Memory Characterization and Performance Analysis at Scale. In *ASPLOS (2)*. ACM, 1203–1217.
[71] Jinshu Liu, Hanchen Xu, Daniel S. Berger, Marcos K. Aguilera, and Huaicheng Li. 2026. Performance Predictability in Heterogeneous Memory. In *ASPLOS (2)*. ACM, 1413–1429.
[72] Shangyu Luo, Zekai J. Gao, Michael N. Gubanov, Luis Leopoldo Perez, and Christopher M. Jermaine. 2017. Scalable Linear Algebra on a Relational Database System. In *ICDE*. IEEE Computer Society, 523–534.
[73] Guido Moerkotte and Thomas Neumann. 2011. Accelerating Queries with Group-By and Join by Groupjoin. *Proc. VLDB Endow.* 4, 11 (2011), 843–851.
[74] Ingo Müller, Peter Sanders, Arnaud Lacurie, Wolfgang Lehner, and Franz Färber. 2015. Cache-Efficient Aggregation: Hashing Is Sorting. In *SIGMOD Conference*. ACM, 1123–1136.
[75] Yusuke Nagasaka, Satoshi Matsuoka, Ariful Azad, and Aydin Buluç. 2018. High-Performance Sparse Matrix-Matrix Products on Intel KNL and Multicore Architectures. In *ICPP Workshops*. ACM, 34:1–34:10.
[76] Yusuke Nagasaka, Satoshi Matsuoka, Ariful Azad, and Aydin Buluç. 2019. Performance optimization, modeling and analysis of sparse matrix-matrix products on multi-core and many-core processors. *Parallel Comput.* 90 (2019).
[77] Tien T. Nguyen, F. Maxwell Harper, Loren Terveen, and Joseph A. Konstan. 2018. User Personality and User Satisfaction with Recommender Systems. *Inf. Syst. Frontiers* 20, 6 (2018), 1173–1189.
[78] Taehyeong Park, Seokwon Kang, Myung-Hwan Jang, Sang-Wook Kim, and Yongjun Park. 2023. Orchestrating Large-Scale SpGEMMs using Dynamic Block Distribution and Data Transfer Minimization on Heterogeneous Systems. In *ICDE*. IEEE, 2456–2459.
[79] Md. Mostofa Ali Patwary, Nadathur Rajagopalan Satish, Narayanan Sundaram, Jongsoo Park, Michael J. Anderson, Satya Gautam Vadlamudi, Dipankar Das, Sergey G. Pudov, Vadim O. Pirogov, and Pradeep Dubey. 2015. Parallel Efficient Sparse Matrix-Matrix Multiplication on Multicore Platforms. In *ISC (Lecture Notes in Computer Science)*, Vol. 9137. Springer, 48–57.
[80] Constantine D. Polychronopoulos and David J. Kuck. 1987. Guided Self-Scheduling: A Practical Scheduling Scheme for Parallel Supercomputers. *IEEE Trans. Computers* 36, 12 (1987), 1425–1439.
[81] Eric Qin, Ananda Samajdar, Hyoukjun Kwon, Vineet Nadella, Sudarshan Srinivasan, Dipankar Das, Bharat Kaul, and Tushar Krishna. 2020. SIGMA: A Sparse and Irregular GEMM Accelerator with Flexible Interconnects for DNN Training. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 58–70. https://doi.org/10.1109/HPCA47549.2020.00015
[82] Moinuddin K. Qureshi. 2014. Memory Scaling is Dead, Long Live Memory Scaling. https://hps.ece.utexas.edu/yale75/qureshi_slides.pdf
[83] Stefan Richter, Victor Alvarez, and Jens Dittrich. 2015. A seven-dimensional analysis of hashing methods and its implications on query processing. *Proceedings of the VLDB Endowment* 9, 3 (2015), 96–107.
[84] Yousef Saad. 2003. *Iterative methods for sparse linear systems*. SIAM.
[85] Tobias Schmidt, Viktor Leis, Peter Boncz, and Thomas Neumann. 2025. SQL-Storm: Taking Database Benchmarking into the LLM Era. *Proc. VLDB Endow.* 18, 11 (2025), 4144–4157.
[86] Stefan Schuh, Xiao Chen, and Jens Dittrich. 2016. An Experimental Comparison of Thirteen Relational Equi-Joins in Main Memory. In *SIGMOD Conference*. ACM, 1961–1976.
[87] Robert Schulze, Tom Schreiber, Ilya Yatsishin, Ryadh Dahimene, and Alexey Milovidov. 2024. ClickHouse - Lightning Fast Analytics for Everyone. *Proc. VLDB Endow.* 17, 12 (2024), 3731–3744.

[88] Malte Skarupke. 2017. I Wrote The Fastest Hashtable. Probably Dance. https://probablydance.com/2017/02/26/i-wrote-the-fastest-hashtable/ Accessed: 2025-02-21.
[89] Ruixuan Sun, Ruoyan Kong, Qiao Jin, and Joseph A. Konstan. 2023. Less Can Be More: Exploring Population Rating Dispositions with Partitioned Models in Recommender Systems. In *UMAP (Adjunct Publication)*. ACM, 291–295.
[90] Wenbo Sun, Asterios Katsifodimos, and Rihan Hai. 2023. Accelerating Machine Learning Queries with Linear Algebra Query Processing. In *SSDBM*. ACM, 13:1–13:12.
[91] Wenbo Sun, Asterios Katsifodimos, and Rihan Hai. 2023. An Empirical Performance Comparison between Matrix Multiplication Join and Hash Join on GPUs. In *ICDEW*. IEEE, 184–190.
[92] Wenbo Sun, Asterios Katsifodimos, and Rihan Hai. 2025. Accelerating machine learning queries with linear algebra query processing. *Distributed Parallel Databases* 43, 1 (2025), 8.
[93] Yizhou Sun, Jiawei Han, Xifeng Yan, Philip S. Yu, and Tianyi Wu. 2011. PathSim: Meta Path-Based Top-K Similarity Search in Heterogeneous Information Networks. *Proc. VLDB Endow.* 4, 11 (2011), 992–1003.
[94] Yan Sun, Yifan Yuan, Zeduo Yu, Reese Kuper, Chihun Song, Jinghan Huang, Houxiang Ji, Siddharth Agarwal, Jiaqi Lou, Ipoom Jeong, Ren Wang, Jung Ho Ahn, Tianyin Xu, and Nam Sung Kim. 2023. Demystifying CXL Memory with Genuine CXL-Ready Systems and Devices. In *MICRO*. ACM, 105–121.
[95] Narayanan Sundaram, Nadathur Satish, Md. Mostofa Ali Patwary, Subramanya Dulloor, Michael J. Anderson, Satya Gautam Vadlamudi, Dipankar Das, and Pradeep Dubey. 2015. GraphMat: High performance graph analytics made productive. *Proc. VLDB Endow.* 8, 11 (2015), 1214–1225.
[96] Jie Tang, Jing Zhang, Limin Yao, Juanzi Li, Li Zhang, and Zhong Su. 2017. DBLP-Citation-Network V10. AMiner Citation Network Dataset, release 10. https://lfs.aminer.cn/lab-datasets/citation/dblp.v10.zip Landing page: https://www.aminer.org/citation. 1.85 GB archive, 4 JSON shards, 3,079,007 papers.
[97] Yupeng Tang, Ping Zhou, Wenhui Zhang, Henry Hu, Qirui Yang, Hao Xiang, Tongping Liu, Jiaxin Shan, Ruoyun Huang, Cheng Zhao, Cheng Chen, Hui Zhang, Fei Liu, Shuai Zhang, Xiaoning Ding, and Jianjun Chen. 2024. Exploring Performance and Cost Optimization with ASIC-Based CXL Memory. In *EuroSys*. ACM, 818–833.
[98] Micron Technology. 2023. Memory Scaling is Dead, Long Live Memory Scaling. https://sg.micron.com/content/dam/micron/global/public/products/white-paper/cxl-impact-dram-bit-growth-white-paper.pdf
[99] The Linux Kernel Development Community. 2024. *Intel(R) Speed Select Technology User Guide*. https://docs.kernel.org/admin-guide/pm/intel-speed-select.html Linux Kernel Documentation.
[100] James D. Trotter, Sinan Ekmekçibasi, Johannes Langguth, Tugba Torun, Emre Düzakin, Aleksandar Ilic, and Didem Unat. 2023. Bringing Order to Sparsity: A Sparse Matrix Reordering Study on Multicore CPUs. In *SC*. ACM, 31:1–31:13.
[101] Aleksandar Vitorovic, Mohammed Elseidy, and Christoph Koch. 2016. Load balancing and skew resilience for parallel joins. In *ICDE*. IEEE Computer Society, 313–324.
[102] Jeffrey Scott Vitter. 1985. Random Sampling with a Reservoir. *ACM Trans. Math. Softw.* 11, 1 (1985), 37–57.
[103] TaiNing Wang and Chee-Yong Chan. 2018. Improving Join Reorderability with Compensation Operators. In *SIGMOD Conference*. ACM, 693–708.
[104] TaiNing Wang and Chee-Yong Chan. 2020. Improved Correlated Sampling for Join Size Estimation. In *ICDE*. IEEE, 325–336.
[105] Marcel Weisgut, Daniel Ritter, Florian Schmeller, Pınar Tözün, and Tilmann Rabl. 2025. CXL-Bench: Benchmarking Shared CXL Memory Access. In *VLDB Workshops*. VLDB.org.
[106] Marcel Weisgut, Daniel Ritter, Pınar Tözün, Lawrence Benson, and Tilmann Rabl. 2025. CXL Memory Performance for In-Memory Data Processing. *Proc. VLDB Endow.* 18, 9 (2025), 3119–3133.
[107] Samuel Williams. 2008. Williams/webbase-1M. https://sparse.tamu.edu/Williams/webbase-1M
[108] Guoqing Xiao, Chuanghui Yin, Tao Zhou, Xueqi Li, Yuedan Chen, and Kenli Li. 2024. A Survey of Accelerating Parallel Sparse Linear Algebra. *ACM Comput. Surv.* 56, 1 (2024), 21:1–21:38.
[109] Weipeng P. Yan and Per-Åke Larson. 1994. Performing Group-By before Join. In *Proceedings of the Tenth International Conference on Data Engineering, February 14-18, 1994, Houston, Texas, USA*. IEEE Computer Society, 89–100.
[110] Weipeng P. Yan and Per-Åke Larson. 1995. Eager Aggregation and Lazy Aggregation. In *VLDB*. Morgan Kaufmann, 345–357.
[111] Abdurrahman Yaşar, Sivasankaran Rajamanickam, Michael Wolf, Jonathan Berry, and Ümit V Çatalyürek. 2018. Fast triangle counting using cilk. In *2018 IEEE High Performance extreme Computing Conference (HPEC)*. IEEE, 1–7.
[112] Mustafa Yazici. 2023. goodbooks 10k the latest. https://www.kaggle.com/datasets/mustafayazici/goodbooks-10k-the-latest. Accessed: 2026-02-21.
[113] Serif Yesil, Azin Heidarshenas, Adam Morrison, and Josep Torrellas. 2020. Speeding up SpMV for power-law graph analytics by enhancing locality & vectorization. In *SC*. IEEE/ACM, 86.

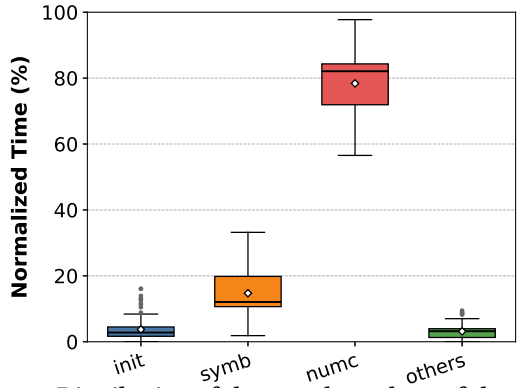


Figure 24: Distribution of the per-phase share of the end-to-end SpGEMM time of AMS-HYB over the 119 SuiteSparse matrices of § 4.1, computed per matrix. `symb` and `numc` denote the symbolic and numeric phases, `init` denotes AMS preparation (morsel construction, cf computation, and morsel sorting), and `others` denotes prefix sum, output allocation, and finalization.

[114] Dongxiang Zhang, Dongsheng Li, Long Guo, and Kian-Lee Tan. 2022. Unsupervised Entity Resolution With Blocking and Graph Algorithms. *IEEE Trans. Knowl. Data Eng.* 34, 3 (2022), 1501–1515.

APPENDIX

A SPGEMM SYMBOLIC PHASE ANALYSIS

AMS targets the numeric phase: its result-size-targeted morsels require the exact per-row output sizes that the symbolic phase discovers, while CALB, SMTact, and HYB address the cache contention, the accumulation, and the output materialization that occur only during numeric execution. The symbolic phase is nevertheless executed in parallel and is shared by the matched baseline and the AMS implementations. Unless a measurement is explicitly labeled as numeric-phase profiling, all reported runtimes and GFLOPS numbers cover the complete SpGEMM pipeline, i.e., symbolic execution, prefix sum and output allocation, AMS preparation, numeric execution, and output finalization. This appendix quantifies how that time is distributed over the phases.

Figure 24 reports the phase composition of AMS-HYB over 119 SuiteSparse matrices. The symbolic phase accounts for an average of 12.07% of the end-to-end runtime, whereas the numeric phase accounts for an average of 82.09%, with AMS preparation (`init`) and the remaining management steps (`others`) contributing only a few percent each. The numeric phase therefore dominates end-to-end execution on real-world matrices, which is where AMS concentrates its optimizations, and the unchanged symbolic phase bounds the attainable end-to-end speedup.

Figure 25 shows the same composition for the matched pair HASH and AMS-HASH on a non-skewed ER workload and on a skewed G500 workload, both at scale 19 and *edgefactor* 16. Since both methods run the identical symbolic implementation, the comparison isolates the effect of AMS on numeric execution: AMS-HASH shrinks the numeric segment, so the symbolic phase and the management steps occupy a visibly larger fraction of a shorter

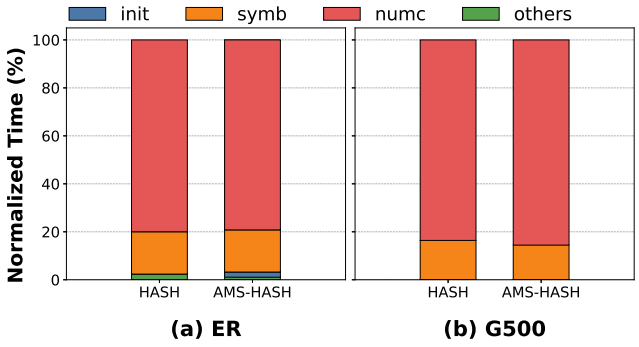


Figure 25: Normalized end-to-end runtime breakdown on ER and G500 at scale 19, edgefactor 16, where every bar is normalized to its own end-to-end SpGEMM time. Each bar is partitioned into the symbolic phase (`symb`), the numeric phase (`numc`), AMS's morsel preparation (`init`), and the remaining management steps of prefix sum, output allocation, and finalization (`others`).

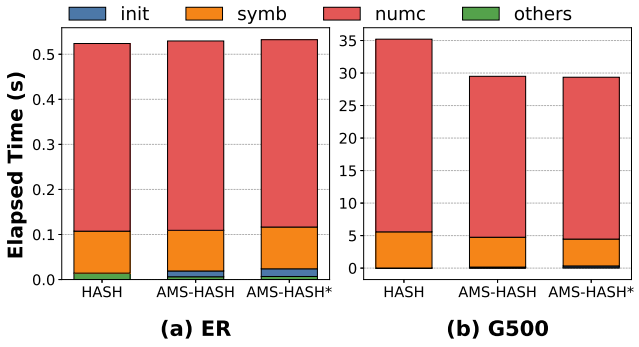


Figure 26: End-to-end runtime breakdown on ER and G500 at scale 19, edgefactor 16. AMS-HASH* is a variant of AMS-HASH that additionally schedules the symbolic phase with CALB, instead of the standard dynamic scheduling with fixed-size morsels used by AMS-HASH. Note the different y-axis ranges of (a) and (b).

end-to-end runtime, while the preparation that AMS adds stays small.

A natural question is whether AMS should be extended to the symbolic phase as well. Figure 26 answers it in absolute time on the same two workloads by adding the variant AMS-HASH*, which also applies CALB to the symbolic phase. On the skewed G500 workload, AMS-HASH reduces the end-to-end time from 35.2 s for HASH to 29.4 s, whereas AMS-HASH* reaches only 29.1 s, a further gain of about 1%: the scheduling shortens the symbolic phase from 4.8 s to 4.5 s, but the extra preparation it requires consumes most of that saving. On the non-skewed ER workload, the variant is in fact marginally slower (0.531 s against 0.528 s), since there is little imbalance to recover and the added preparation is pure overhead. Moreover, applying AMS's output-aware partitioning before the output sizes are known would require either predicting them or performing an additional pass. We therefore scope AMS to the numeric phase and regard symbolic-phase optimization as complementary work rather than assume that its cost is negligible.

Table 7: Number of the 23 evaluated configurations per generator whose widest output row exceeds a given capacity.

widest output row of C exceeds	G500	SSCA
the morsel target	22/23	21/23
half of the L2 capacity	19/23	17/23
the full L2 capacity	13/23	11/23

B OVERSIZED ROWS AND INCREASING SKEW

AMS forms morsels against a target output footprint (§ 3.1). Two questions follow naturally from that design. First, what happens when a single row produces more output than the target, or indeed more than the whole L2 cache, so that the morsel containing it cannot respect the budget? Second, does AMS degrade as matrix skew increases, since skew is what produces such rows? This appendix answers both over the full R-MAT sweep. The short answers are that oversized rows are the common case rather than a corner case, that crossing a cache boundary does not degrade AMS, and that the width of the widest row is not the variable that governs AMS's margin.

B.1 Oversized Rows Are the Common Case

Recall from § 3.1 that the morsel size is a soft target: rows are appended whole, and a row whose own output exceeds the target becomes a morsel by itself. Table 7 counts how often this happens across the 23 evaluated configurations per generator. A majority of configurations contain a row whose output exceeds the entire L2 capacity, and nearly all exceed the morsel target. The extremes are far beyond the target: the widest row of `g500_20_16` holds 623.0K nonzeros (9.51 MB, 4.75× L2), and that of `ssca_20_16` holds 657.1K (10.03 MB, 5.01× L2). From scale 19 upward the widest row accounts for 99–100% of the largest morsel, i.e., the largest morsel simply *is* that row, so its footprint is fixed by the sparsity structure of C and cannot be recovered by retuning the target. Every headline number reported in § 4.2 was therefore already measured in the presence of oversized rows.

B.2 Crossing the L2 Boundary Does Not Degrade AMS

The cleanest test holds matrix dimension and density fixed and varies only the widest row. SSCA and G500 at the same *scale* and *edgefactor* are produced by the same generator invocation with identical vertex and edge counts, and differ only in the R-MAT probabilities, so G500 is the more skewed of the two and is the side whose widest row crosses the boundary. Four configurations straddle a cache boundary in this way, two at half of L2 and two at the full L2, and Table 8 reports them.

In all four pairs the oversized-row side obtains the *larger* speedup, by 0.23, 0.16, 0.21, and 0.12. The most informative column is the HASH baseline, which is essentially flat across each pair (0.591

Table 8: Matched pairs that straddle a cache boundary. Each pair shares *scale* and *edgefactor* and differs only in the R-MAT probabilities. Throughput in GFLOPS; the widest output row is given relative to the 2 MB private L2.

		widest row	AMS-HASH	HASH	speedup
18-2	SSCA	0.46×L2 (fits)	0.612	0.591	1.04×
	G500	0.55×L2 (exceeds)	0.760	0.599	**1.27×**
19-1	SSCA	0.43×L2 (fits)	0.545	0.542	1.01×
	G500	0.61×L2 (exceeds)	0.661	0.569	**1.16×**
19-2	SSCA	0.84×L2 (fits)	0.730	0.593	1.23×
	G500	1.02×L2 (exceeds)	0.860	0.598	**1.44×**
20-1	SSCA	0.78×L2 (fits)	0.683	0.557	1.23×
	G500	1.15×L2 (exceeds)	0.745	0.553	**1.35×**

against 0.599, 0.542 against 0.569, 0.593 against 0.598, and 0.557 against 0.553 GFLOPS). Crossing the cache boundary therefore costs the baseline nothing measurable, and the entire difference within a pair comes from AMS gaining rather than from the baseline losing. Whatever an unsplittable oversized row costs AMS in straggler terms, the additional skew that produced that row returns more.

We state two honest caveats. The two sides of a pair also differ in overall skew, which is precisely what AMS exploits, so this compares "oversized row and more skew" against "row fits and less skew" rather than isolating row width alone. It nonetheless answers the question at hand, because the oversized-row side is never the loser. In addition, G500's $nnz(C)$ is 1.8–2.2× larger in each pair, so the oversized side also performs more work; throughput is flop-normalized, so this does not mechanically favor either side.

B.3 Density, Not Row Width, Governs the Margin

The complementary view holds density fixed and grows *scale*, letting the widest row sweep across the cache boundaries within a single generator. Table 9 reports the widest row and the AMS-HASH speedup for every configuration of the sweep.

The sign of the trend flips with density. At *edgefactor* ≥ 8 the advantage narrows monotonically as the widest row grows (G500 at *edgefactor* 8: 1.81×, 1.44×, 1.32×, 1.20×, with $\rho = -0.98$), whereas at *edgefactor* ≤ 2 it *rises* ($\rho = +1.00$ and $+0.93$). Over all 42 matched configurations the correlation between the widest row's footprint and the speedup is only $\rho = +0.28$. The width of the widest row is therefore not the controlling variable; density is. SSCA reproduces the same pattern, so this is not a G500 artifact.

Where the margin does narrow at high density, we do not attribute it to the L2 crossing, for two reasons. First, along the *edgefactor* 8 ladder the largest single drop on G500 falls at scale $17 \rightarrow 18$ while on SSCA it falls at scale $18 \rightarrow 19$, even though both generators emit the same edge count at a given configuration and their operands therefore leave the LLC at the same step. No single sharp cache crossing explains the narrowing in both. Second, two effects unrelated to row width shrink the fraction of runtime that AMS can attack. The operand B grows from roughly 34 MB to 269 MB across that ladder and thus falls out of the 60 MB LLC, turning each gather in Gustavson's inner loop into a DRAM random access; and $nnz(C)$ grows about 20×, so output materialization, a fixed cost no scheduler can remove, takes an increasing share of runtime while cf drifts down from 2.52 to 2.17. Both effects reduce the share of runtime spent on contention, which is the only component AMS addresses. We note that this is a latency effect rather than a bandwidth wall: achieved output write bandwidth *falls* from 8.6 to 4.9 GB/s against the platform's 179.50 GB/s peak, so the kernel stalls on random gathers rather than saturating the bus. Separating the two explanations would require L2 and LLC miss counters along this ladder, which we leave to future measurement.

The claim we do make is the bounded one. Across all 20 configurations whose widest row exceeds the full L2 capacity, AMS-HASH outperforms the matched HASH baseline in every case, with a minimum of 1.20× and a geometric mean of 1.40×. The speedup never approaches, let alone crosses, parity. Along the G500 *edgefactor* 8 ladder the speedup factor falls by 34% (1.81× to 1.20×); expressed instead as excess over parity, the margin falls by 76% (+81% to +20%). The two framings are not interchangeable, and we quote the former throughout.

B.4 Limitation

AMS performs *inter*-row scheduling. A single row that dominated the total computation would set the critical path for AMS and equally for every other row-parallel method, since such a row cannot be divided without intra-row parallelism. In our workloads the widest row accounts for at most 0.16% of $nnz(C)$, so this regime does not arise, but we identify intra-row parallelism and segmented accumulation as complementary techniques for it.

Table 9: Widest output row relative to the 2 MB L2 and the corresponding AMS-HASH speedup over HASH, for every evaluated configuration. Each cell reads [×L2, speedup]. ρ is the Spearman correlation between the two within a row.

	G500					SSCA			
edgefactor	scale 17	scale 18	scale 19	scale 20	ρ	scale 17	scale 18	scale 19	scale 20
1	0.18, 0.85×	0.34, 0.93×	0.61, 1.16×	1.15, 1.35×	+1.00	0.13, 0.78×	0.24, 0.92×	0.43, 1.01×	0.78, 1.23×
2	0.29, 1.08×	0.55, 1.27×	1.02, 1.44×	1.89, 1.45×	+0.93	0.25, 0.84×	0.46, 1.04×	0.84, 1.23×	1.51, 1.40×
4	0.43, 1.54×	0.80, 1.63×	1.50, 1.47×	2.81, 1.22×	−0.21	0.41, 1.22×	0.76, 1.42×	1.39, 1.53×	2.57, 1.43×
8	0.56, 1.81×	1.06, 1.44×	2.02, 1.32×	3.80, 1.20×	−0.98	0.57, 1.73×	1.08, 1.64×	2.03, 1.29×	3.78, 1.24×
16	0.69, 1.65×	1.31, 1.49×	2.50, 1.24×	—	−0.83	0.72, 1.91×	1.38, 1.56×	2.65, 1.34×	—
32	0.79, 1.71×	1.52, 1.45×	—	—	−0.60	0.84, 1.88×	1.63, 1.56×	—	—